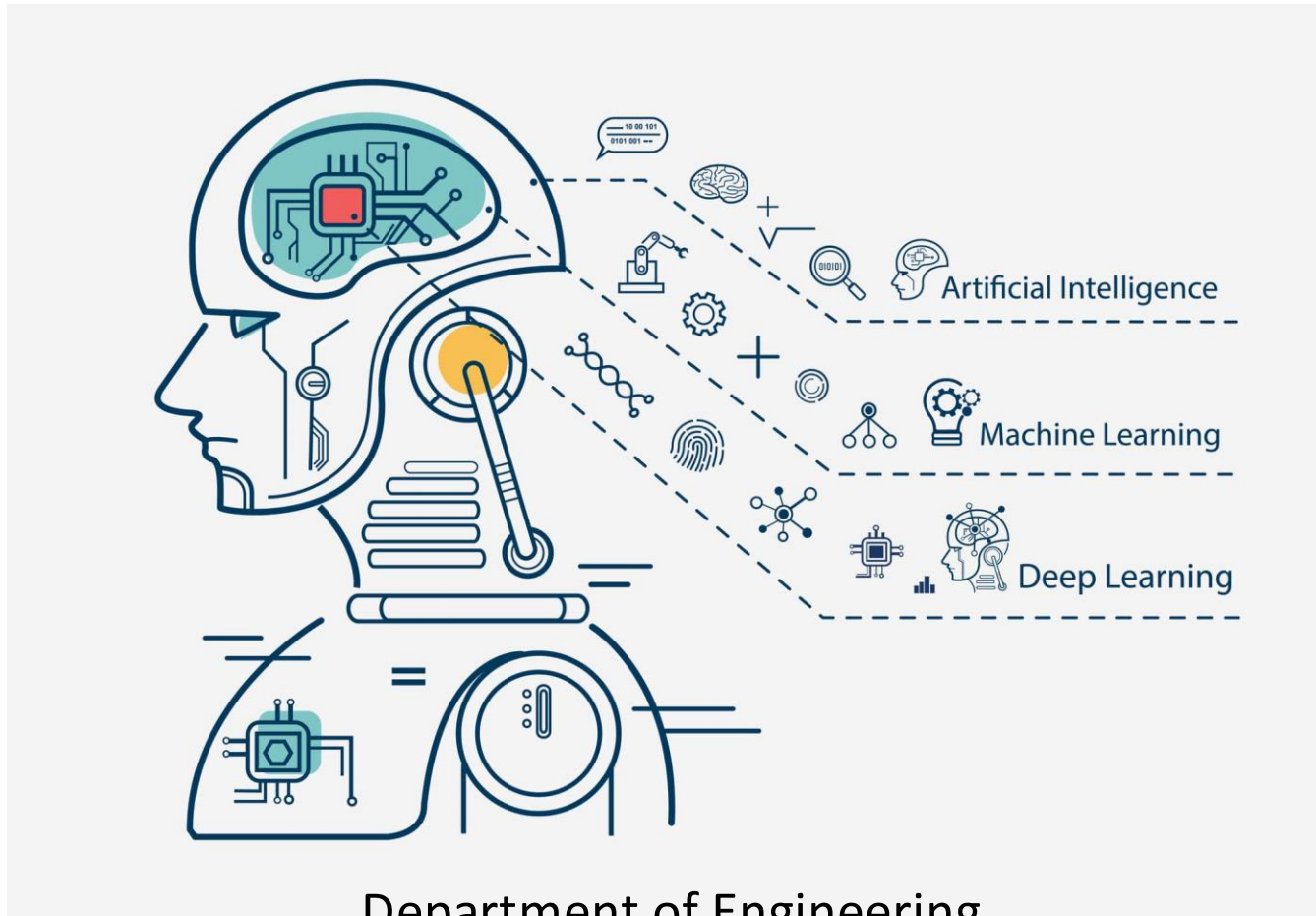


ECE612 Applied Machine Learning Engineering

Lecture 2

CNN

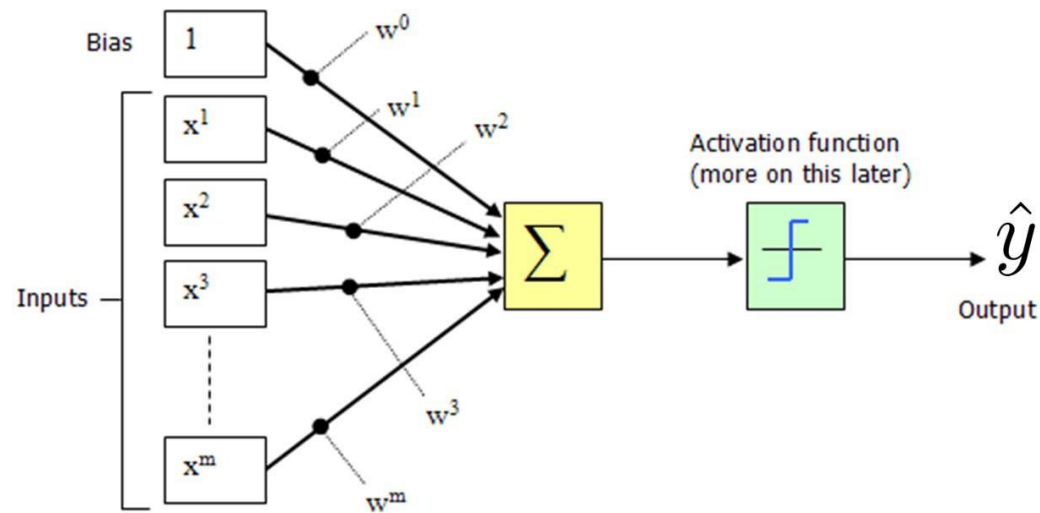


Department of Engineering
Drexel University
Summer 2025

Artificial Neural Networks & Backpropagation

Perceptron (Visually)

- m-dimensional input
- 1 output, $\hat{y}$
- m+1 weights to learn
 - bias term + weights for each of m dimensions

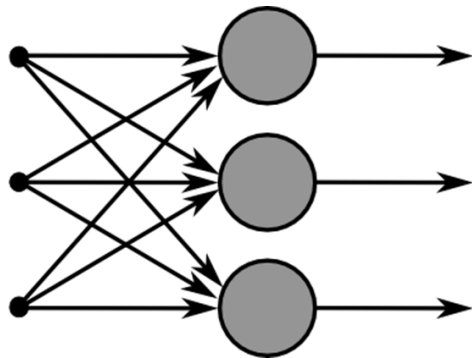


Threshold Units

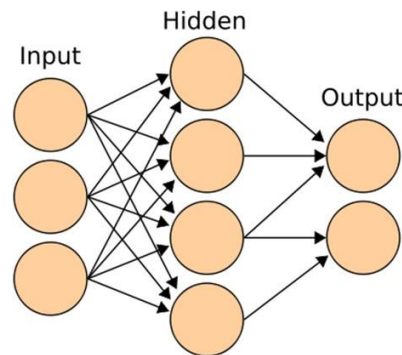
- Simple perceptrons $f(x) = \begin{cases} 1 & \text{if } w \cdot x + b > 0 \\ 0 & \text{otherwise} \end{cases}$
- Very simple artificial neurons
- Note output is 0/1 instead of -1/+1
 - Corresponds to the notion of neuron “firing”

Multi-Layer Networks

- Single perceptrons cannot compute certain simple functions, like XOR
- We can overcome this using multiple layers



One-Layer Perceptron



Two-Layer Perceptron

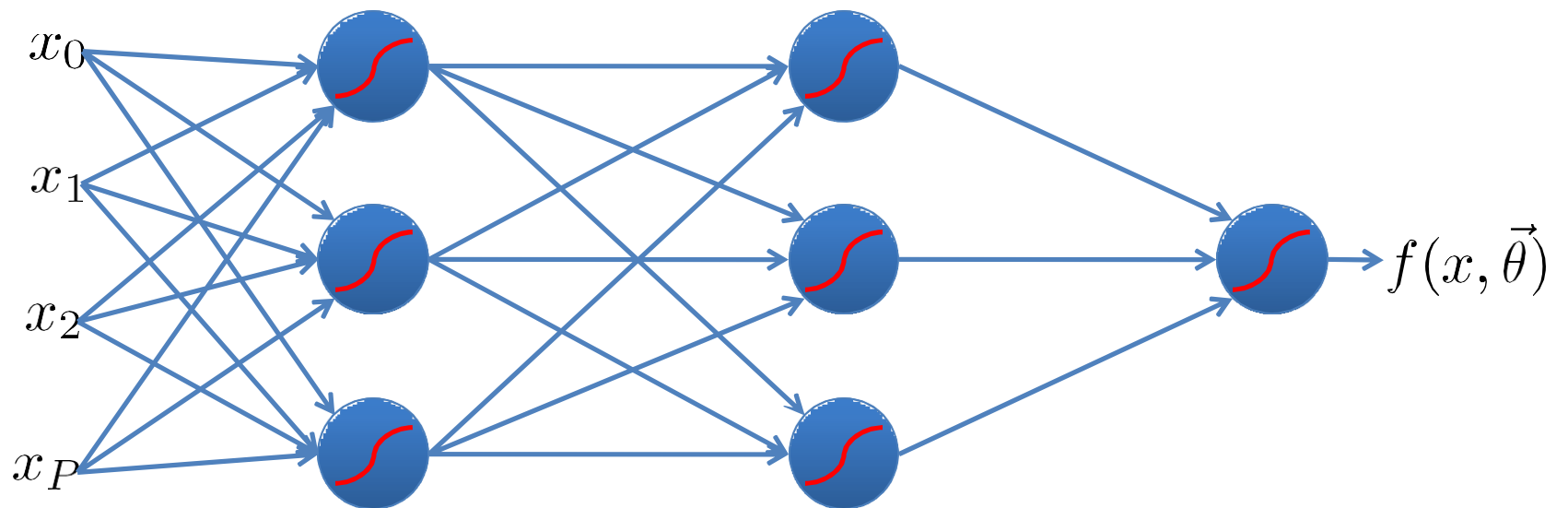
Number of layers may or may not count input layer (usually doesn't)

Feature / Representation Learning

- Each layer maps input to a new feature space
 - You can get away with less clever feature representations using MLP
 - At what cost?
 - Increased VC dimension (also MLP prone to overfitting)
- Nonlinearities can make interpretation of the hidden layers **very** difficult
- Often, MLPs are treated as **black boxes**

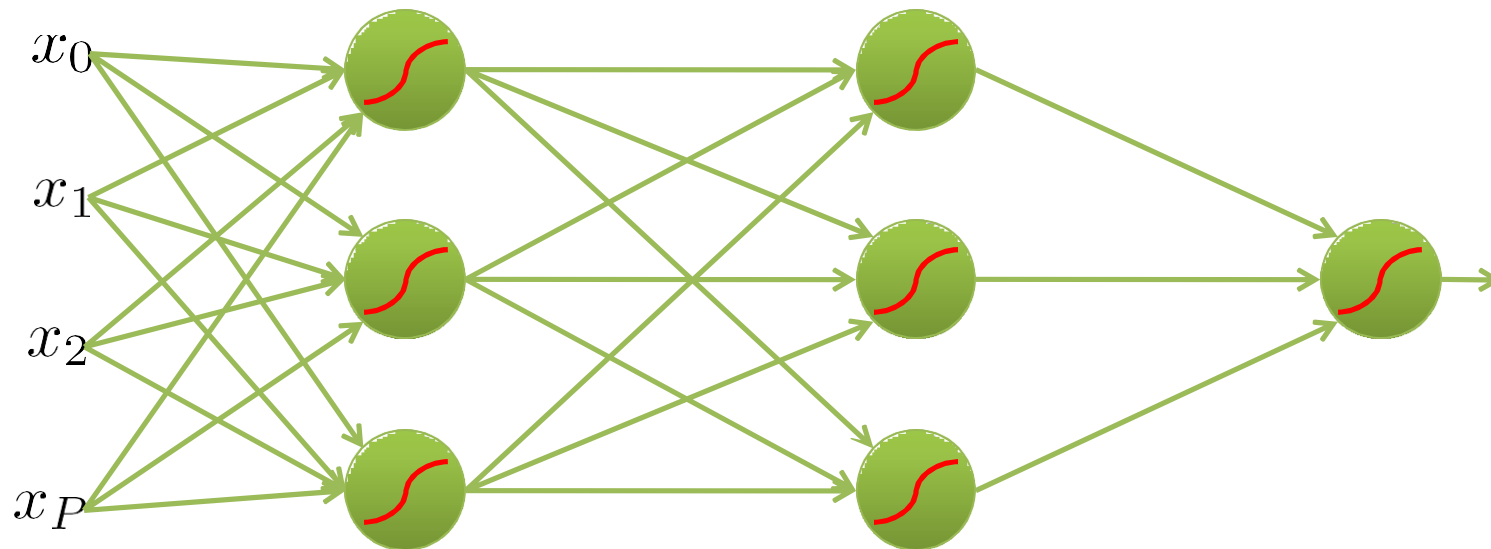
Multilayer Networks

- Cascade Neurons together
- The output from one layer is the input to the next
- Each Layer has its own sets of weights



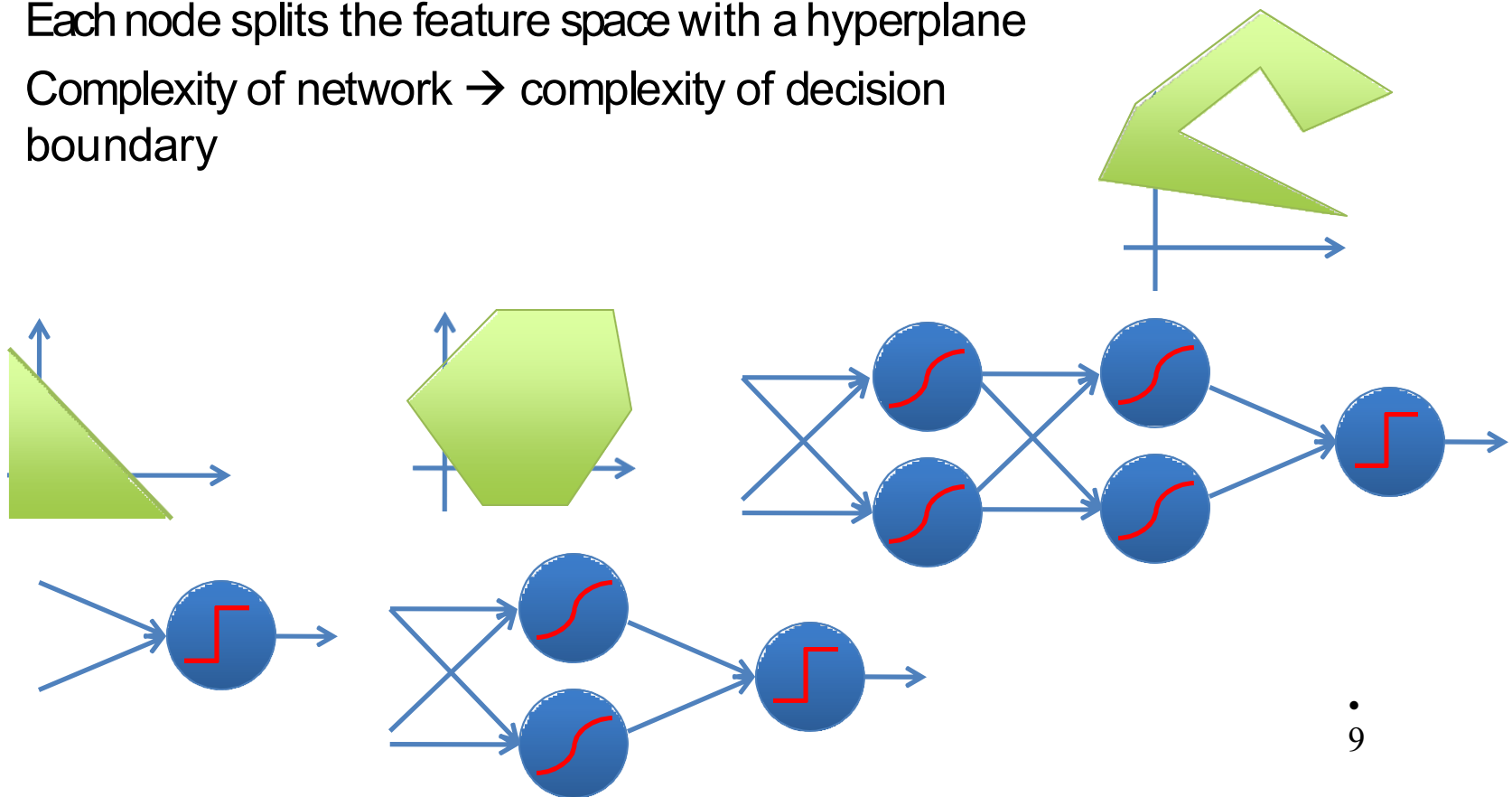
FeedForward Networks

- Predictions are fed forward through the network to classify



Multilayer Networks in Feature Space

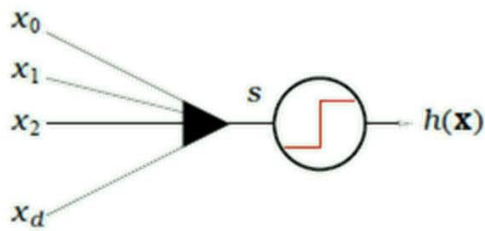
- More layers can handle more complicated spaces
 - but require more parameters
- Each node splits the feature space with a hyperplane
- Complexity of network $\rightarrow$ complexity of decision boundary



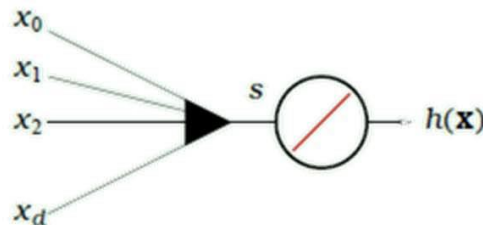
Many Types of Units

- Perceptrons can do more than thresholding
 - Linear Regression
 - Logistic Regression
 - Many other nonlinear, differentiable functions
 - We'll come back to this

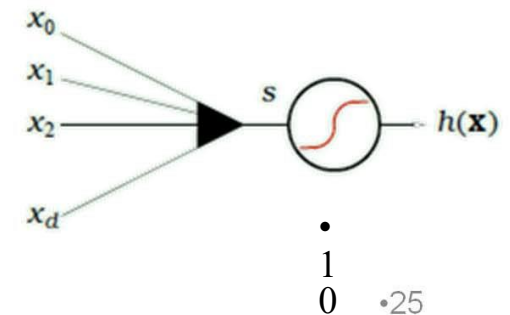
$$h(\mathbf{x}) = \text{sign}(s)$$



$$h(\mathbf{x}) = s$$



$$h(\mathbf{x}) = \theta(s)$$



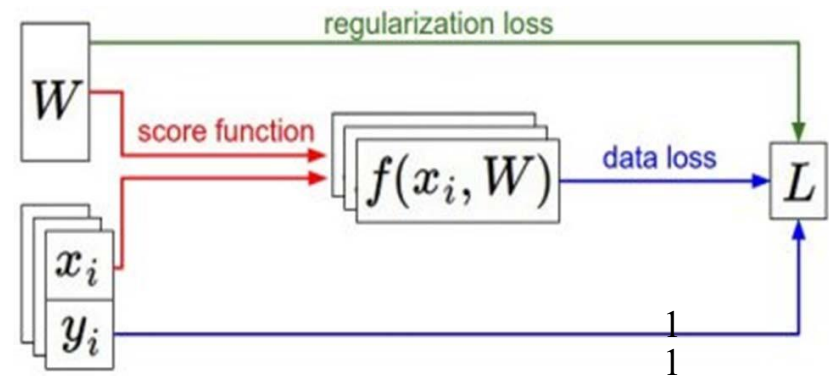
Generic Perceptron Training

- We have some **dataset**: (X, Y)
- We have a **score function**: $s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$
- We have a **loss function**
 - with (possibly) regularization
- **Gradient Descent**

Hinge: $l(y, y') = \max(0, 1 - yy')$

Exponential: $l(y, y') = \exp(-yy')$

Squared loss: $l(y, y') = (y - y')^2$



Gradient Descent

- pick a starting point (w)
- repeat until loss doesn't decrease in any dimension:
 - pick a dimension
 - move a small amount in that dimension towards decreasing loss (using the derivative)

$$w_j = w_j - \eta \frac{d}{dw_j} \text{loss}(w)$$

- Computing this derivative?

Numerical Gradient

- In 1-dimension, the **derivative** of a function:

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

- In multiple dimensions, the **gradient** is the vector of partial derivatives

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[-2.5,
?,
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

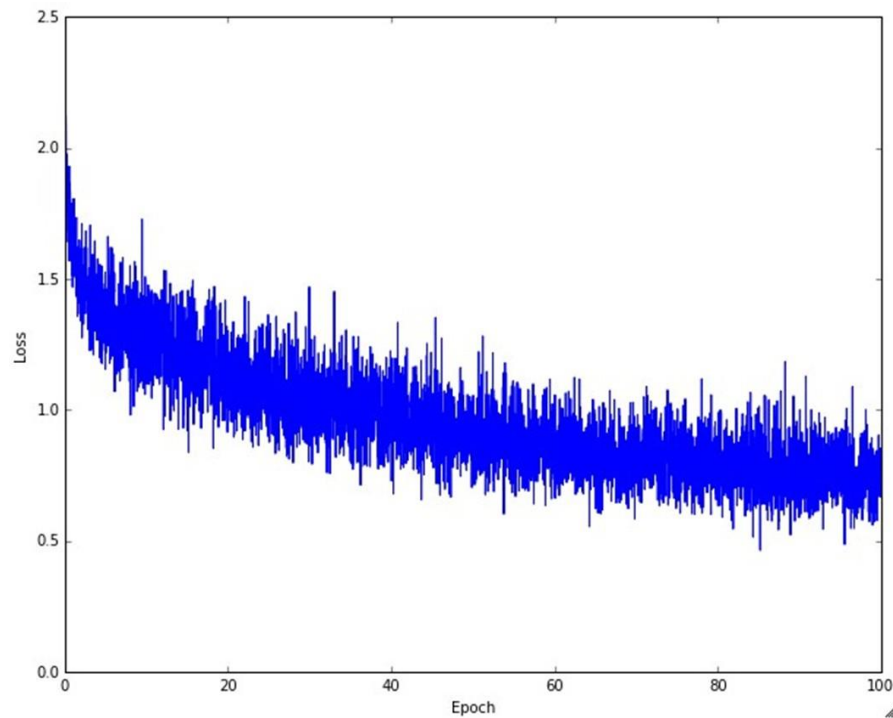
?,
?,...]

Mini-batch Gradient Descent

- Only use a small portion of the training set to compute the gradient

```
# Vanilla Minibatch Gradient Descent  
  
while True:  
    data_batch = sample_training_data(data, 256) # sample 256 examples  
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)  
    weights += - step_size * weights_grad # perform parameter update
```

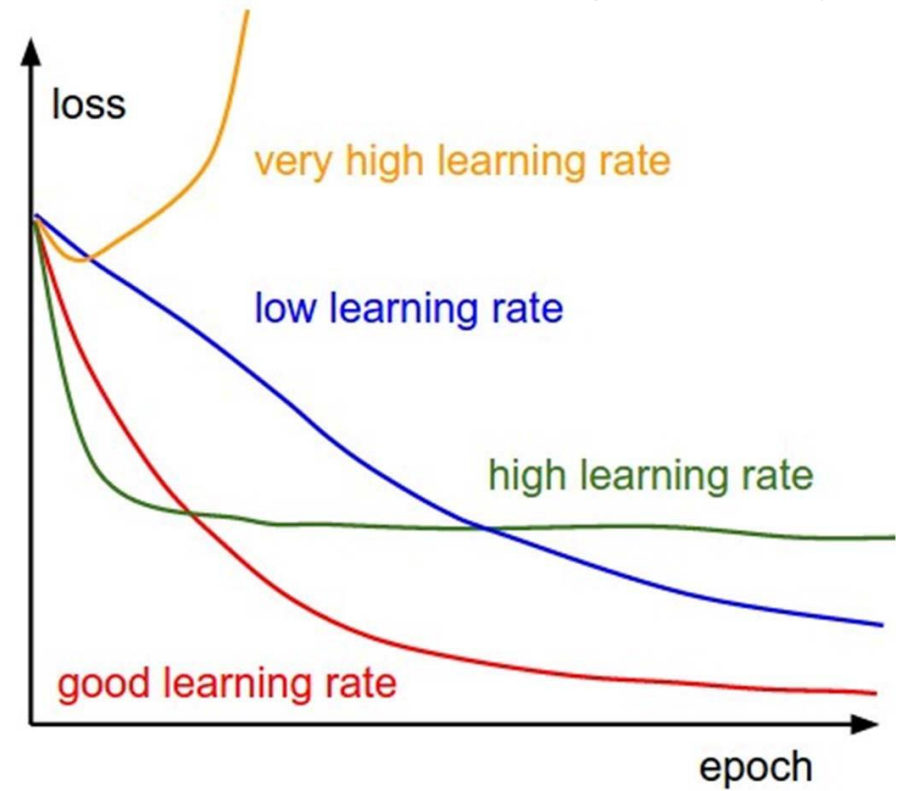
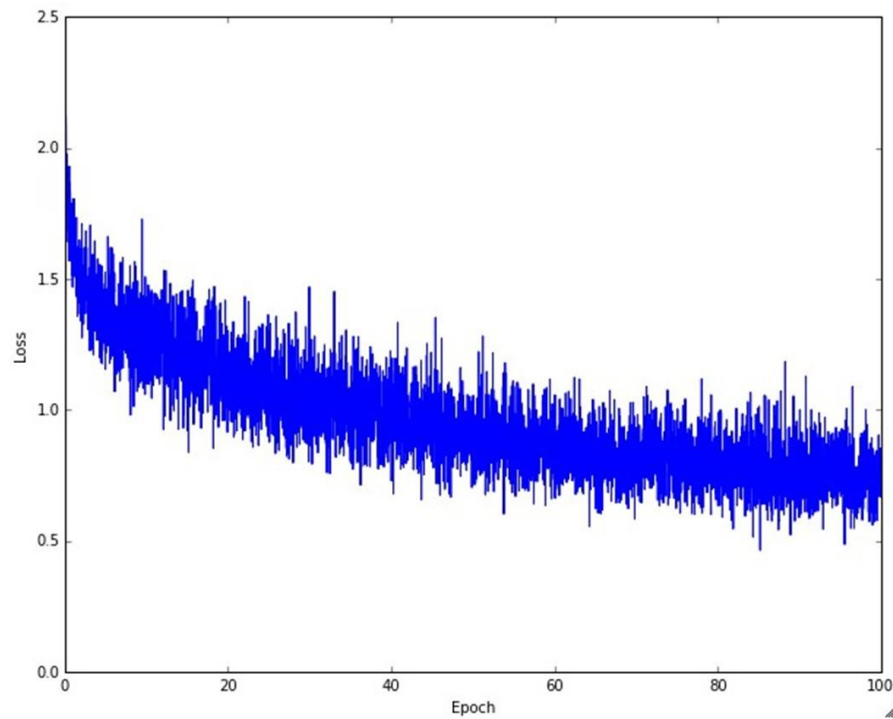
- Common mini-batch sizes are powers of 2 (e.g., 32, 64, 256)
- Most common approach to training NN



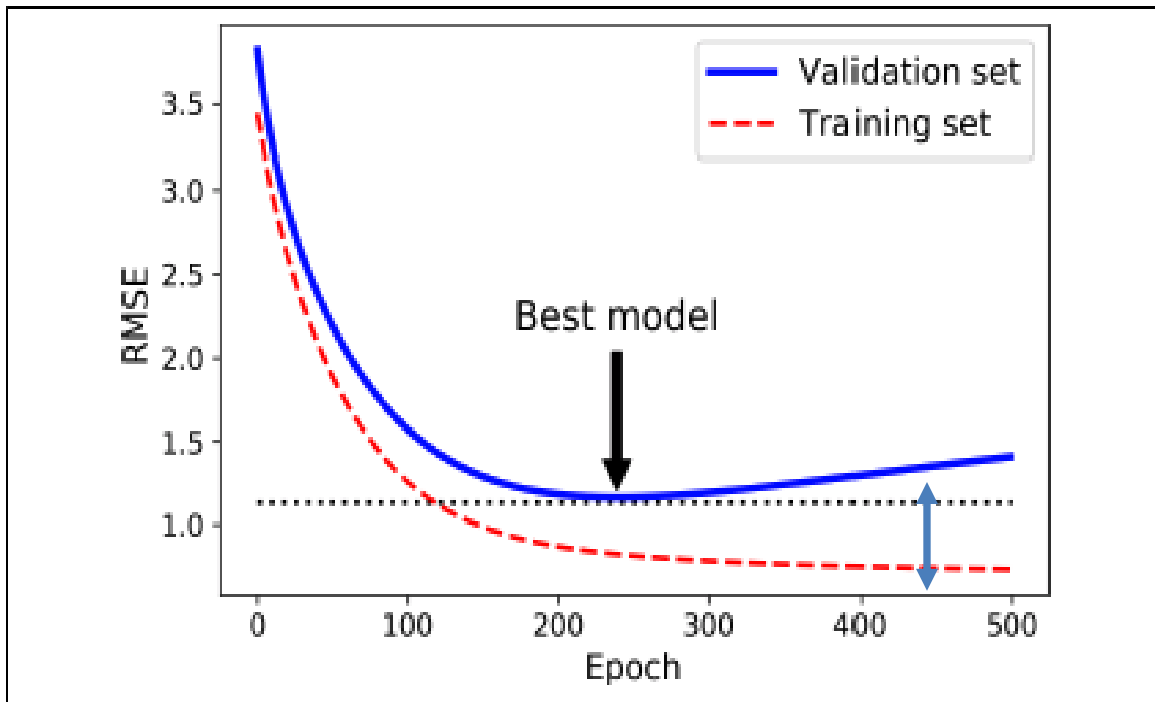
Example of optimization progress while training a neural network.

(Loss over mini-batches goes down over time.)

The effects of step size (or “learning rate



Combating Overfitting with Early Stopping



big gap = overfitting

Early stopping is a form of regularization used to avoid overfitting during the training of a machine learning model.

Figure 4-20. Early stopping regularization

- **Handling Bumpy Validation Loss** (e.g. stoch./mini-batch): keep track of the minimum validation loss encountered during training. When stopping the training, revert to the parameters from the epoch with the minimum validation loss to ensure the best model is selected.
- Set a **patience parameter** that allows the training to continue for a specified number of iterations after the minimum validation loss is reached. If the validation loss does not improve within this patience period, the training process should be stopped to prevent overfitting.

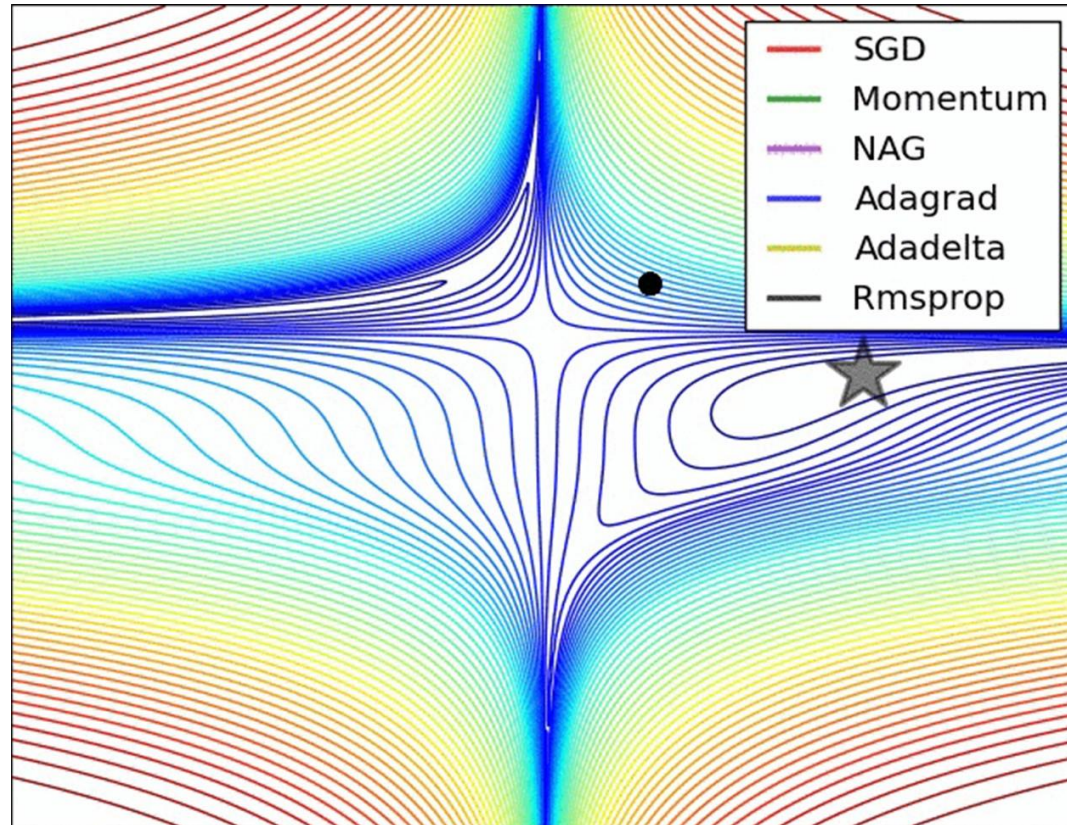
Weight Update Step

```
# Vanilla Minibatch Gradient Descent

while True:
    data_batch = sample_training_data(data, 256) # sample 256 examples
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
    weights += - step_size * weights_grad # perform parameter update
```

- Can do something fancier than this vanilla update
 - Many algorithms (momentum, Adagrad, RMSProp, Adam, ...)

The effects of different update formulas



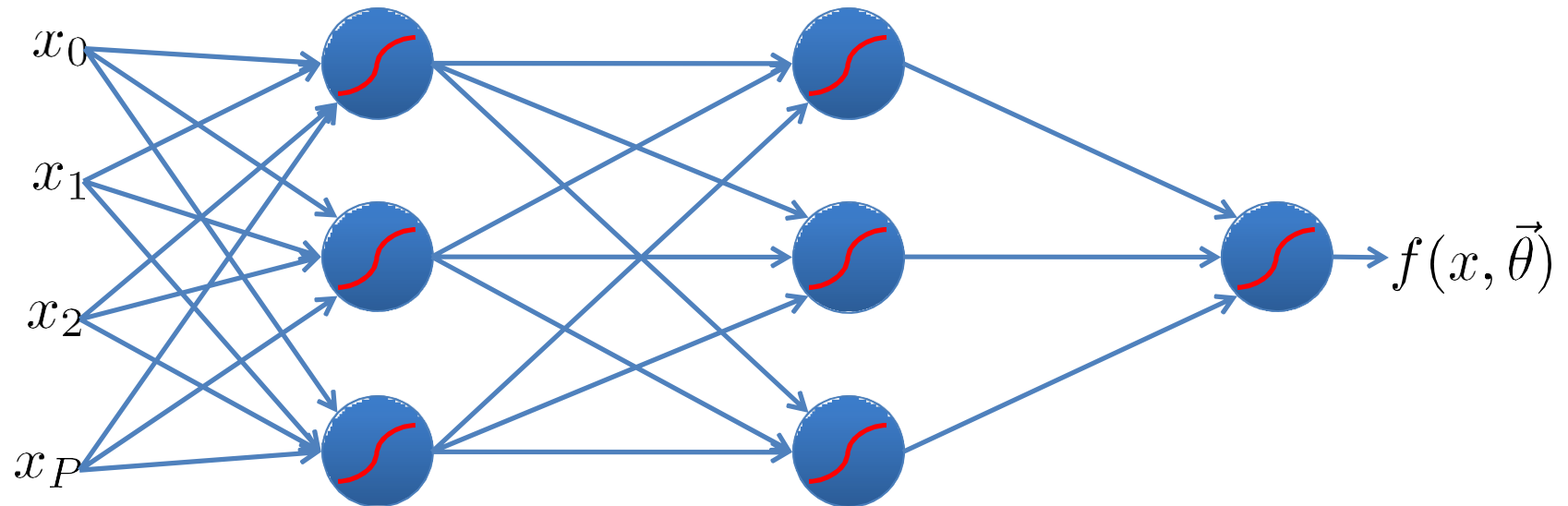
(image credits to Alec Radford)

What do we have so far...

- Components – biological plausibility
 - Neuron / node
 - Synapses / weight
- Perceptrons / Artificial Neurons
 - Threshold Units (boolean functions / classification)
 - Linear Units (regression)
 - Nonlinear Units
- Similarities across units...
 - All can be trained via gradient descent
 - Weights of the connections between nodes determine the behavior of a unit or network

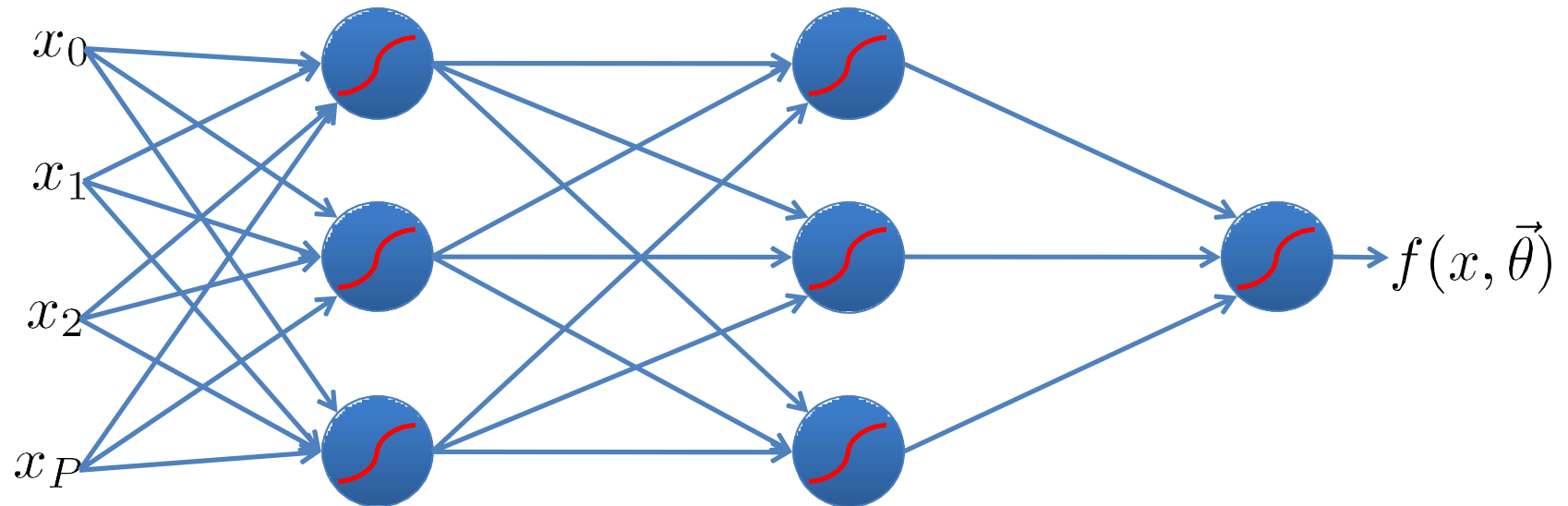
How Do We Train MLPs?

- Need to learn weights for all of the edges
- PLA/GD works for 1-layer network
 - We will (slightly) modify this for MLP



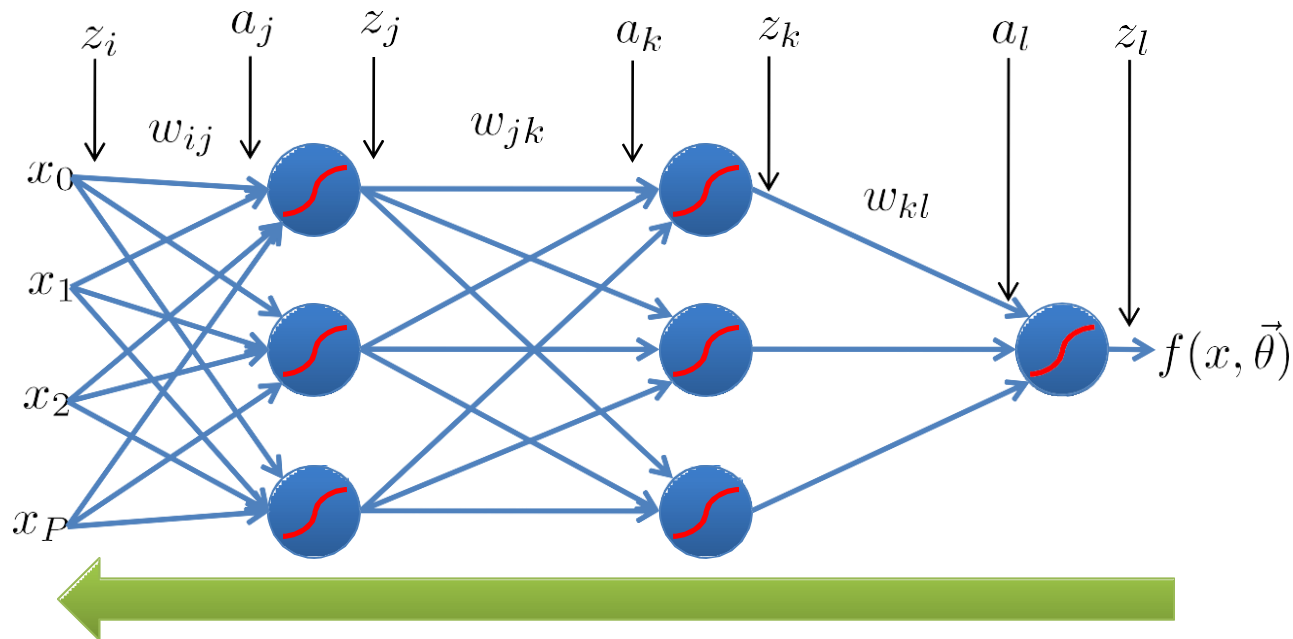
Error Backpropagation

- Gradient descent on the whole network
- Training will proceed ***backwards***
 - From the last layer to the first



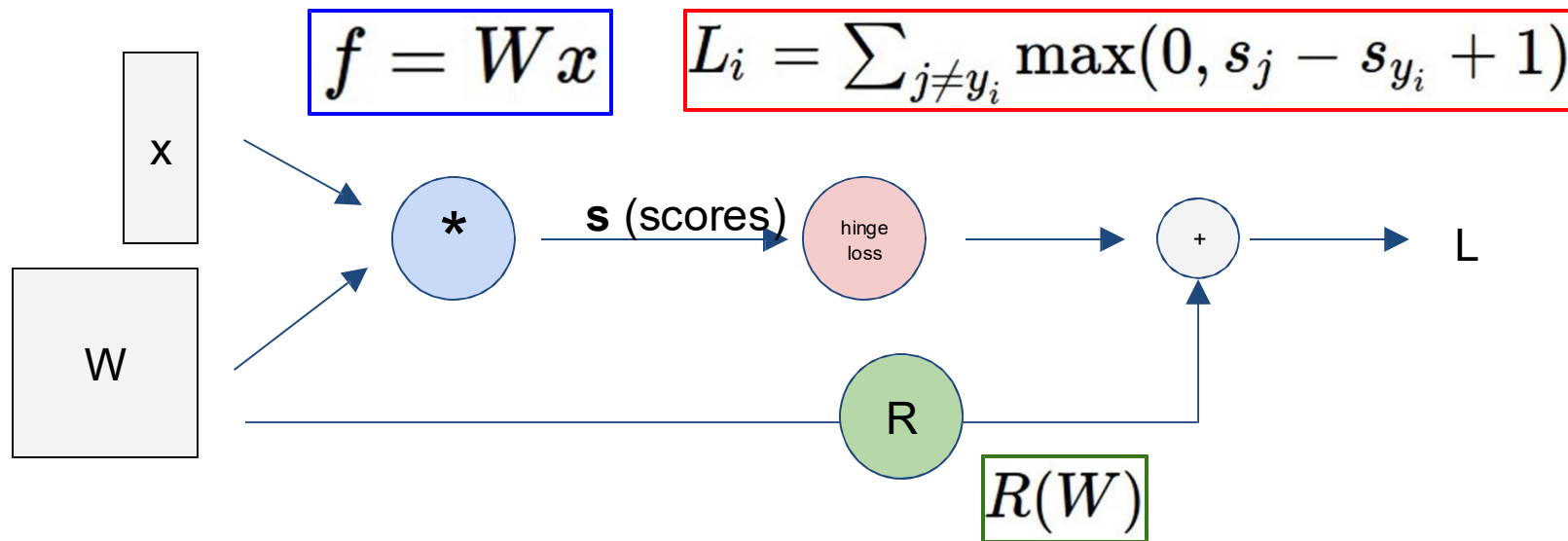
Error Backpropagation

- Error backprop unravels the multivariate chain rule and solves the gradient for each partial component separately
- The target values for each layer come from the next layer
- This ***feeds the errors back*** along the network

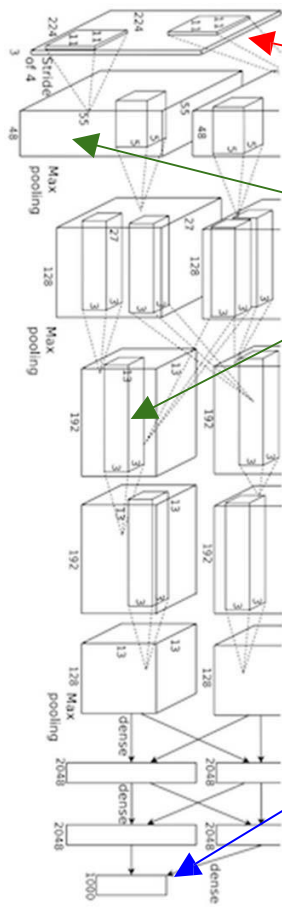


Neural Network Computational Graph

- Consider a neural network as a large computational graph



Larger Computational Graphs

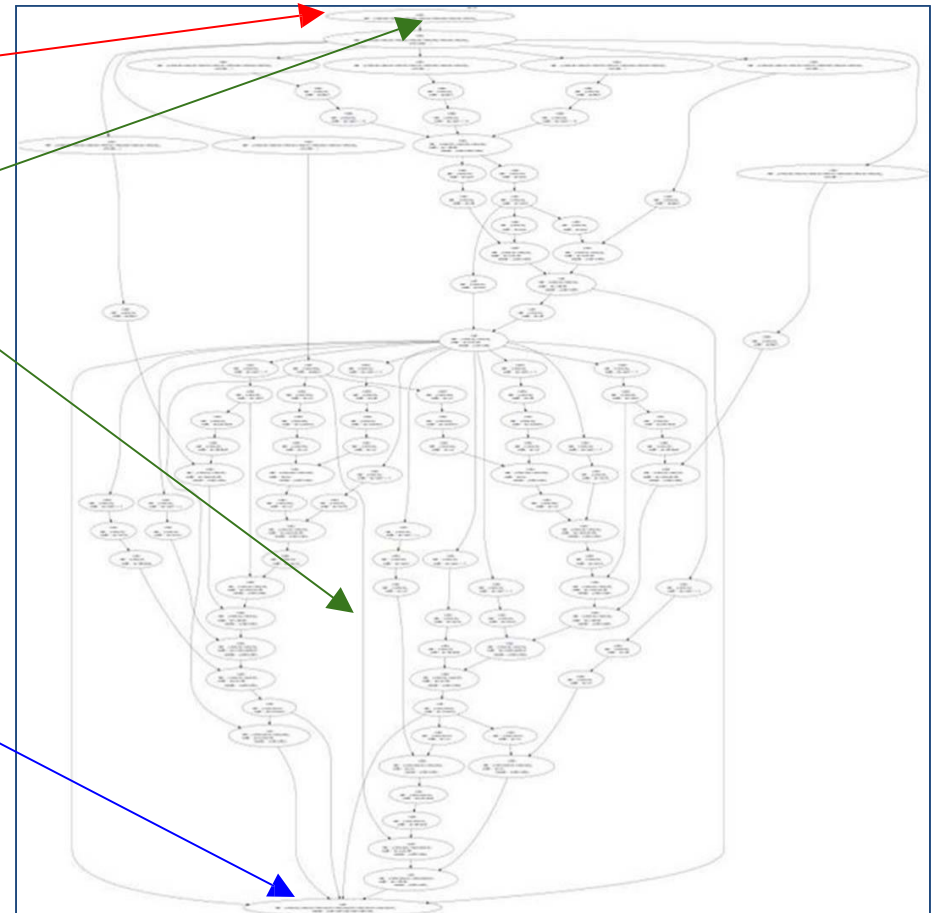


AlexNet

input

weights

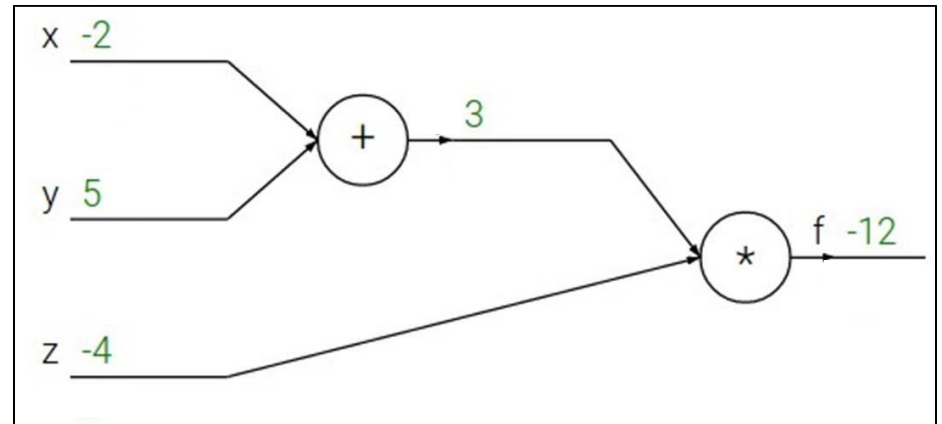
loss



Neural Turing Machine

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



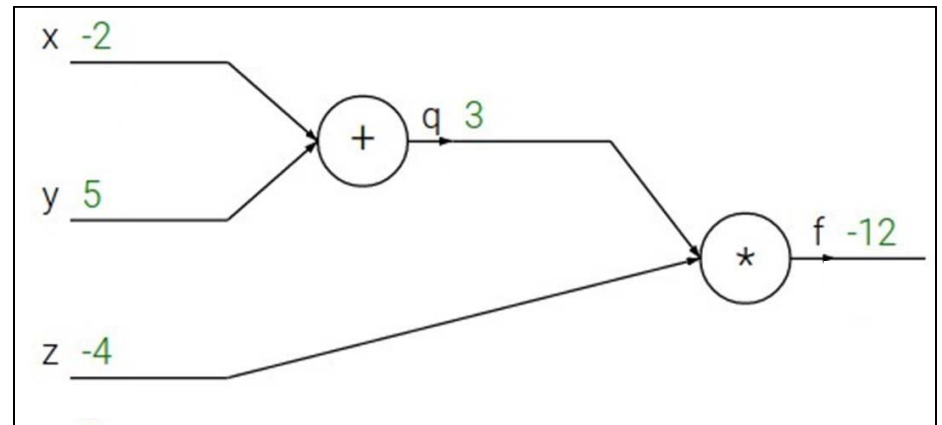
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



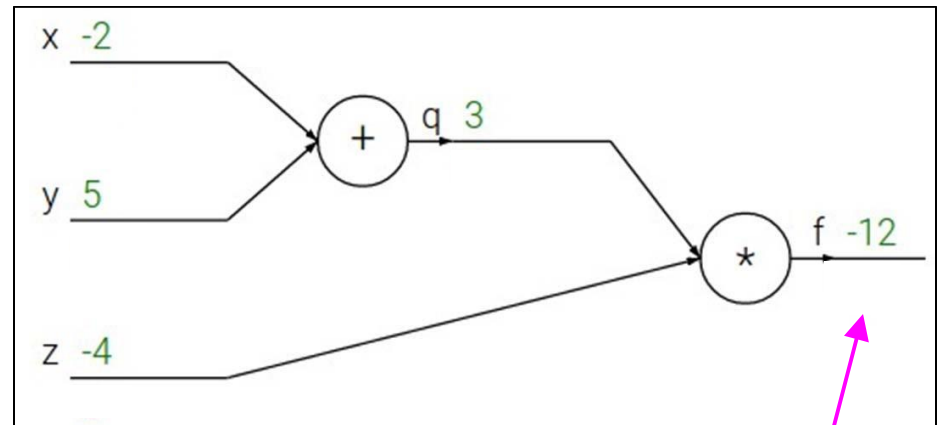
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial f}$$

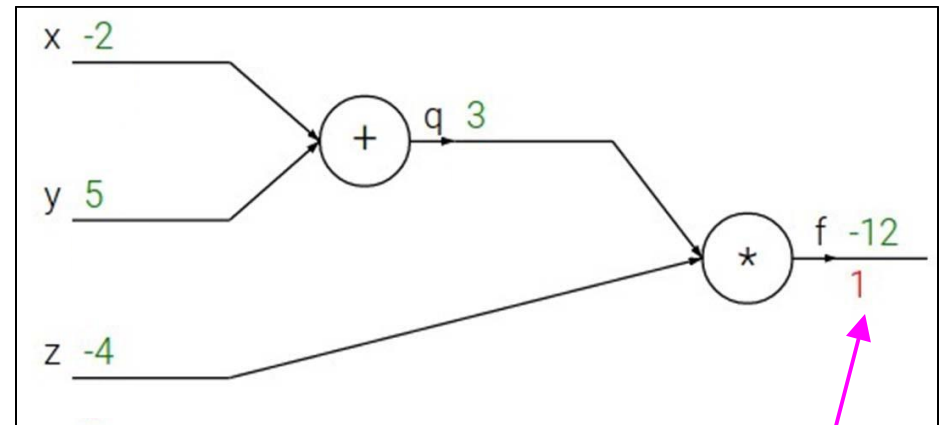
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial f}$$

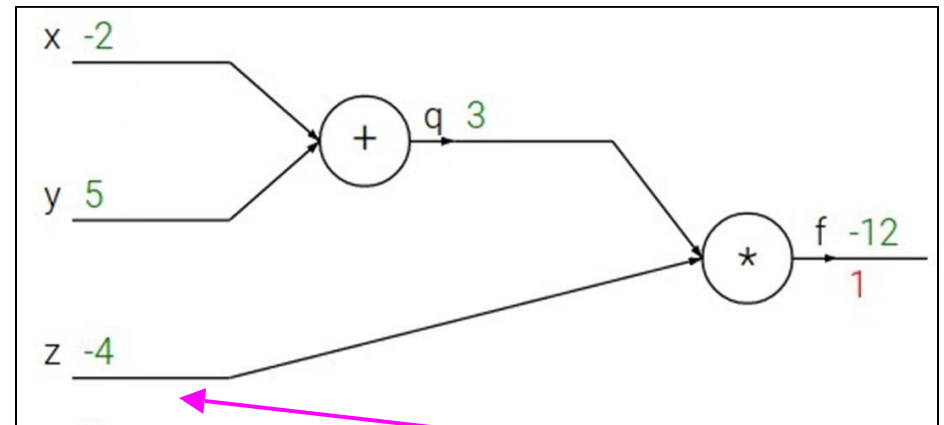
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial z}$$

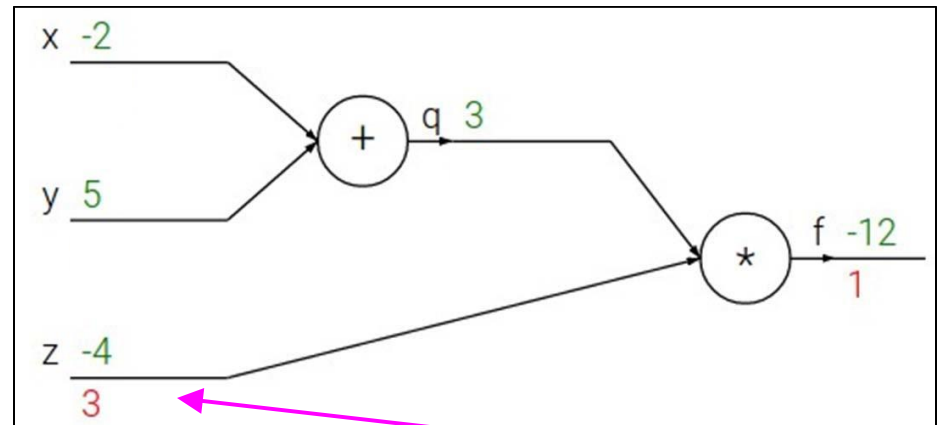
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial z}$$

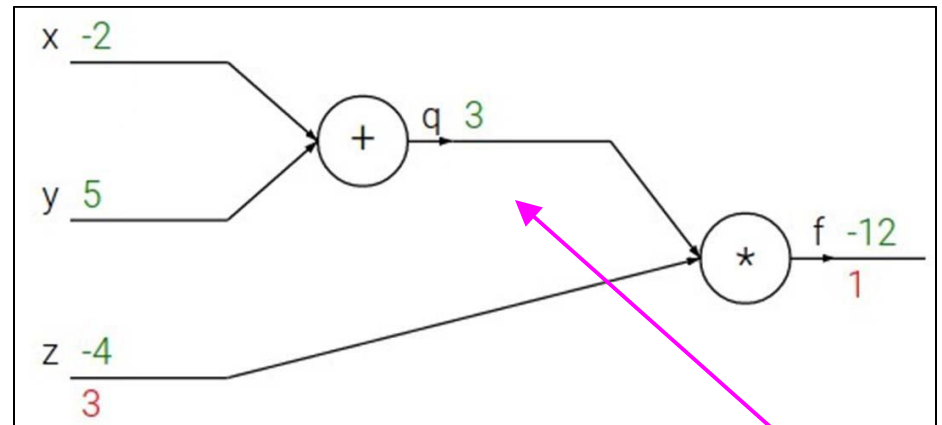
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial q}$$

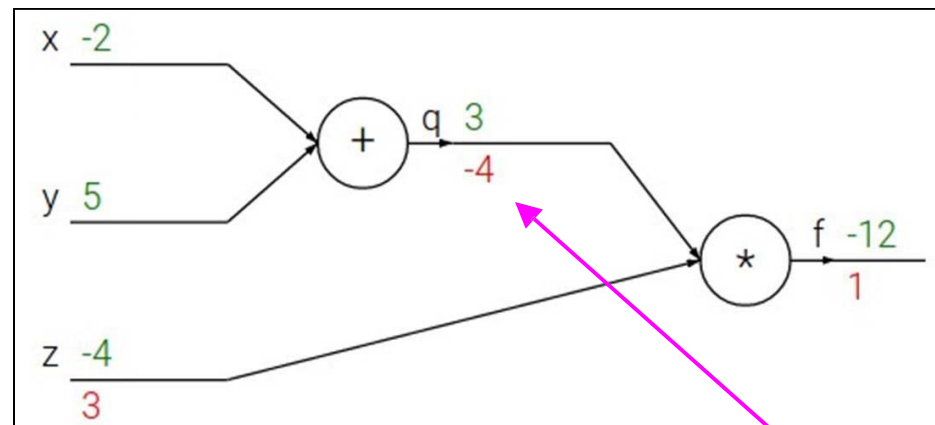
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial q}$$

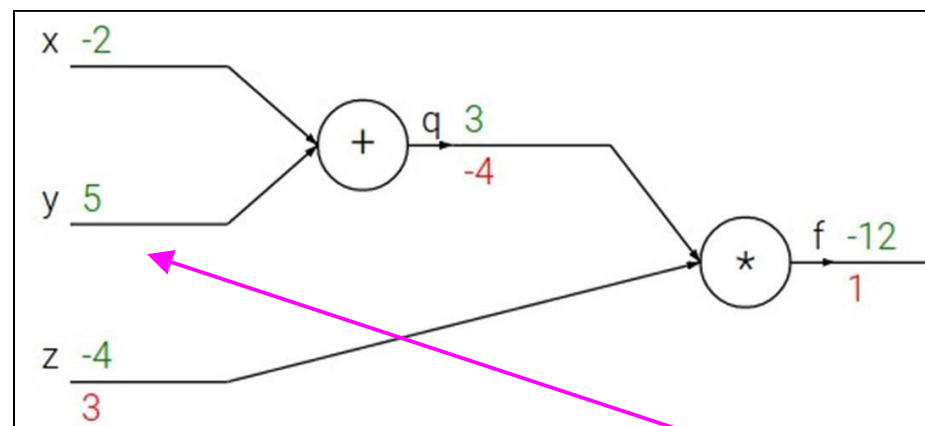
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



$$\frac{\partial f}{\partial y}$$

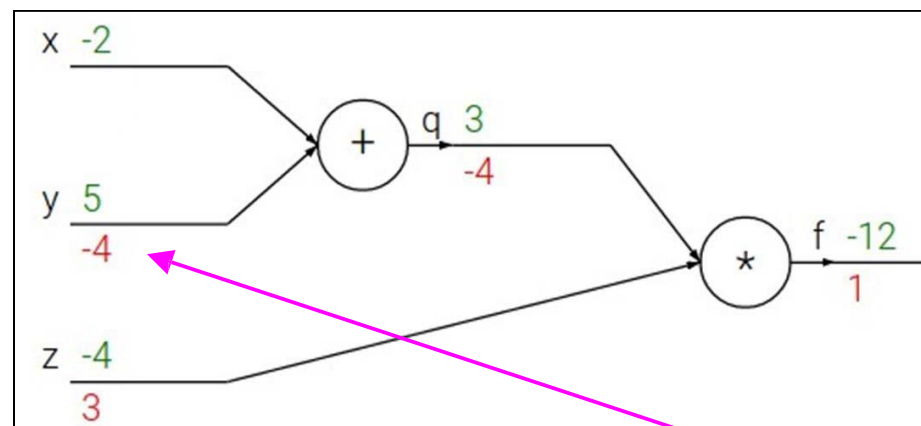
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

$$\frac{\partial f}{\partial y}$$

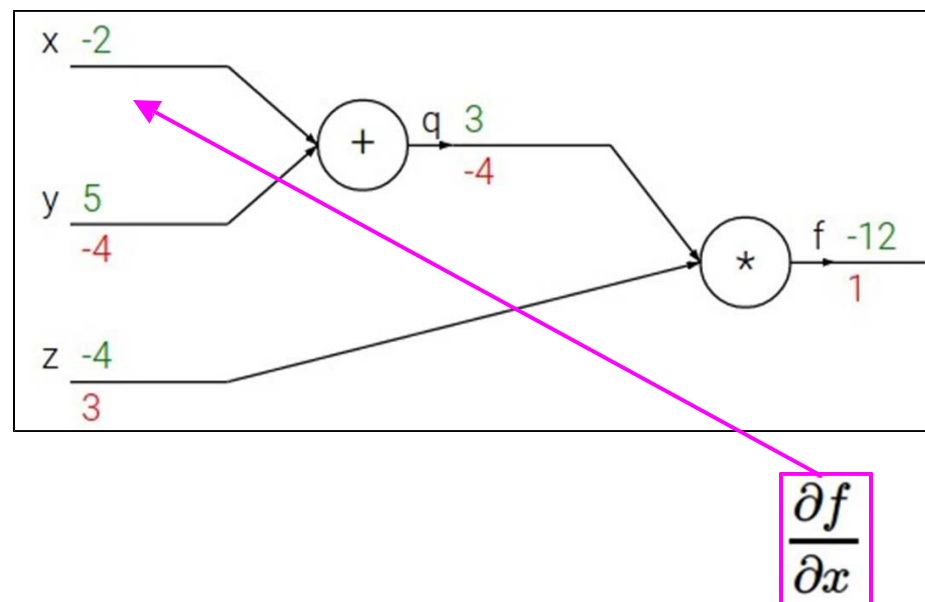
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



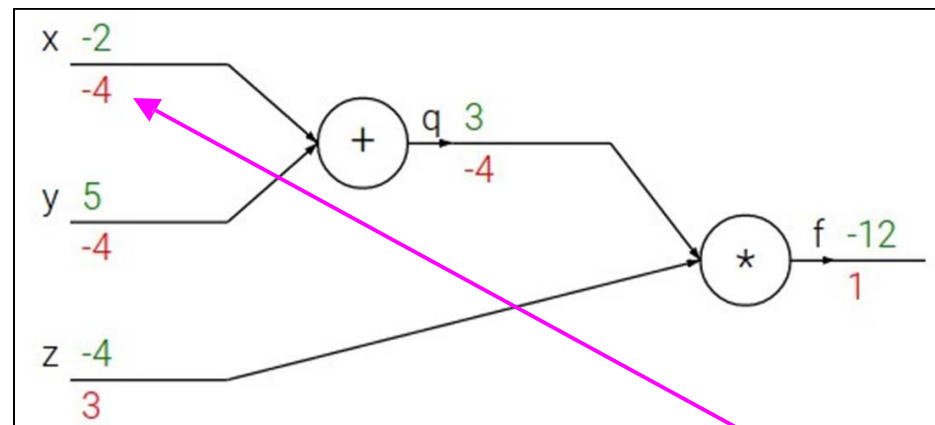
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

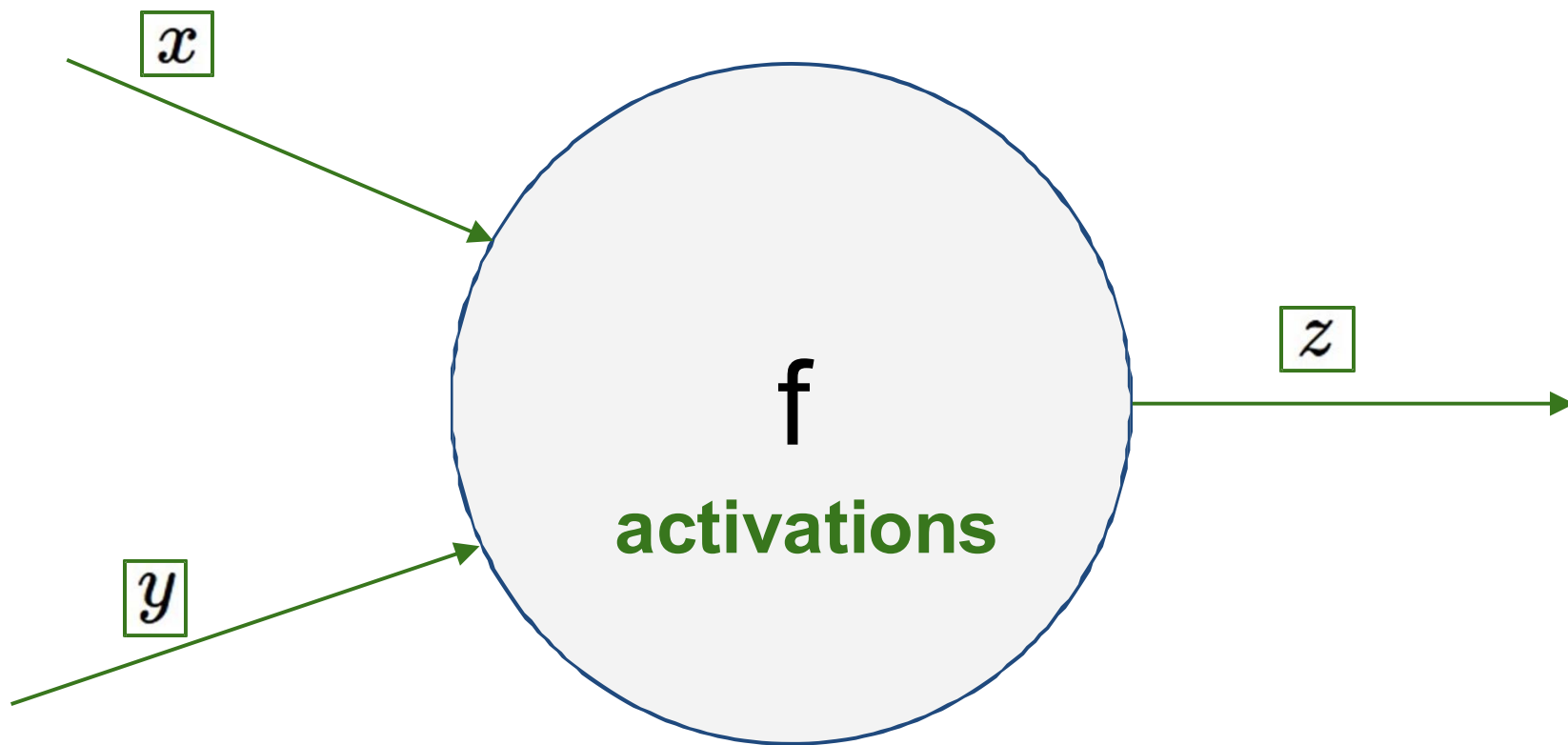
$$\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$$



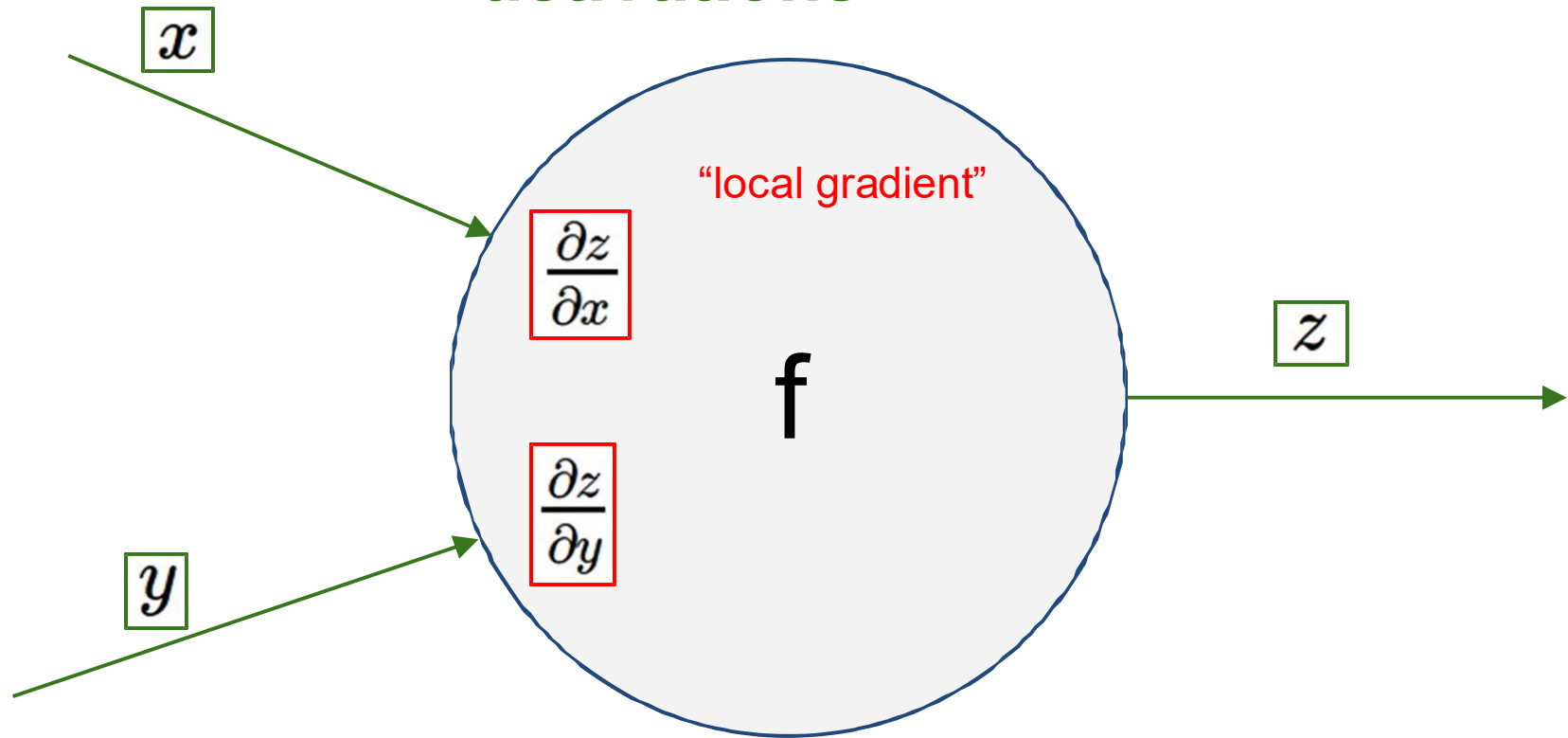
$$\frac{\partial f}{\partial x}$$

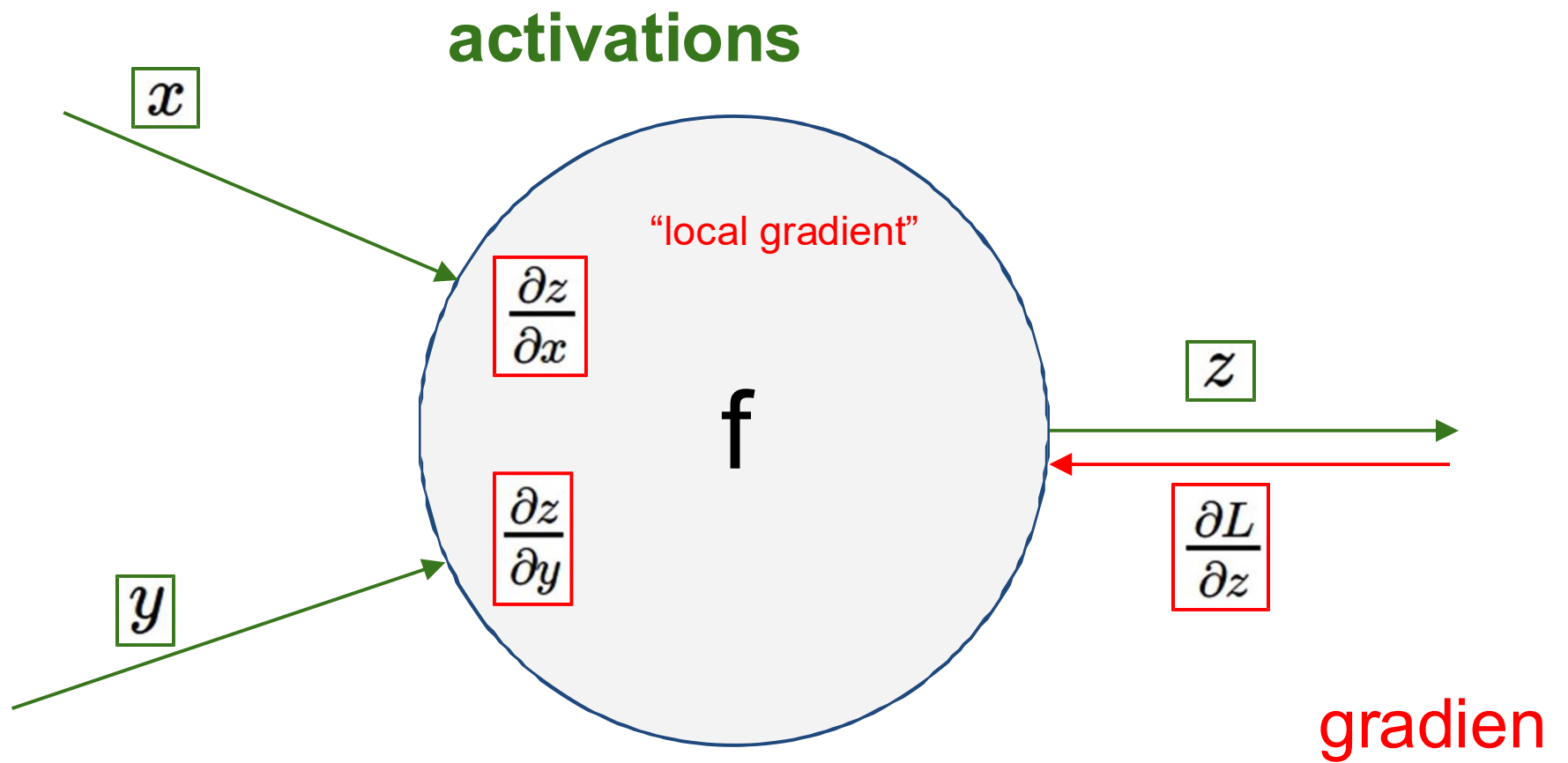
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

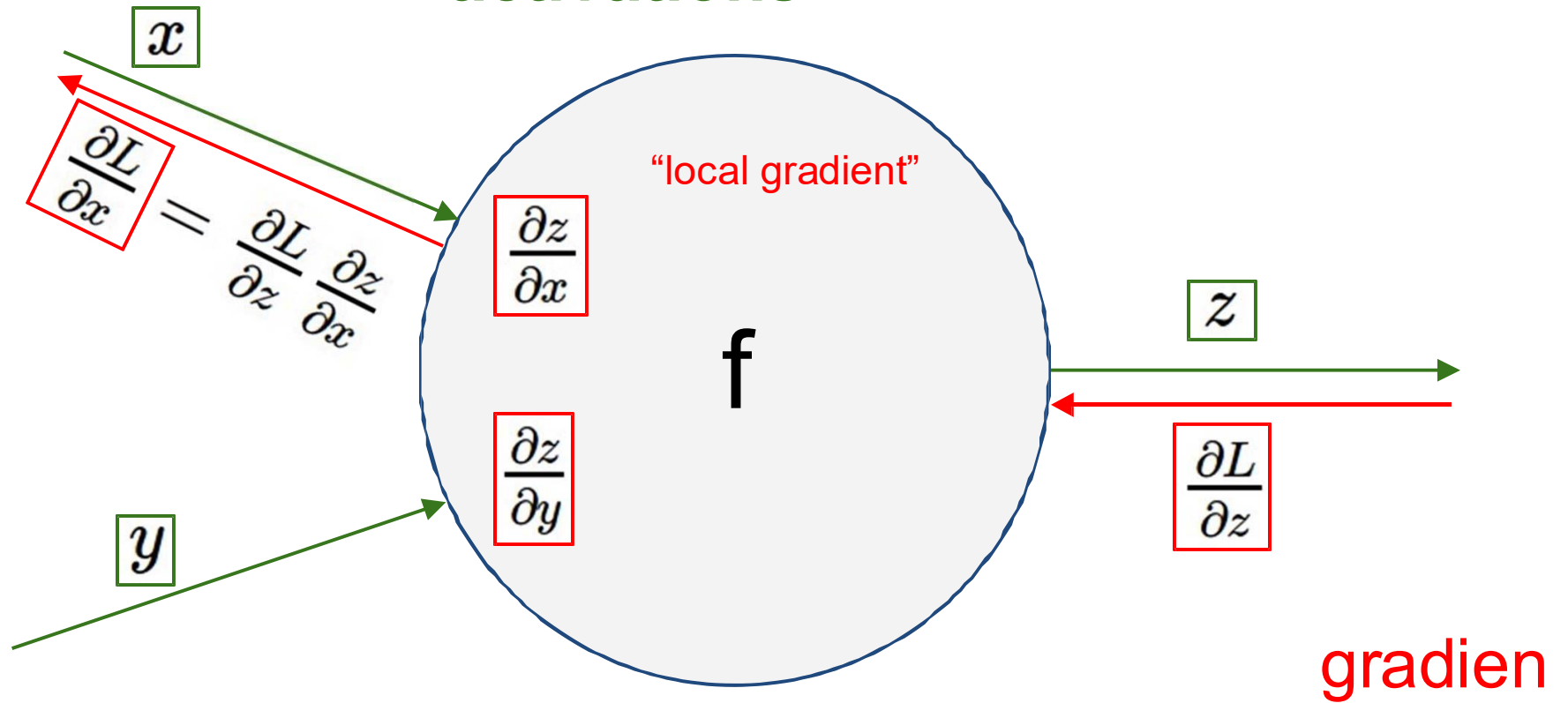


activations

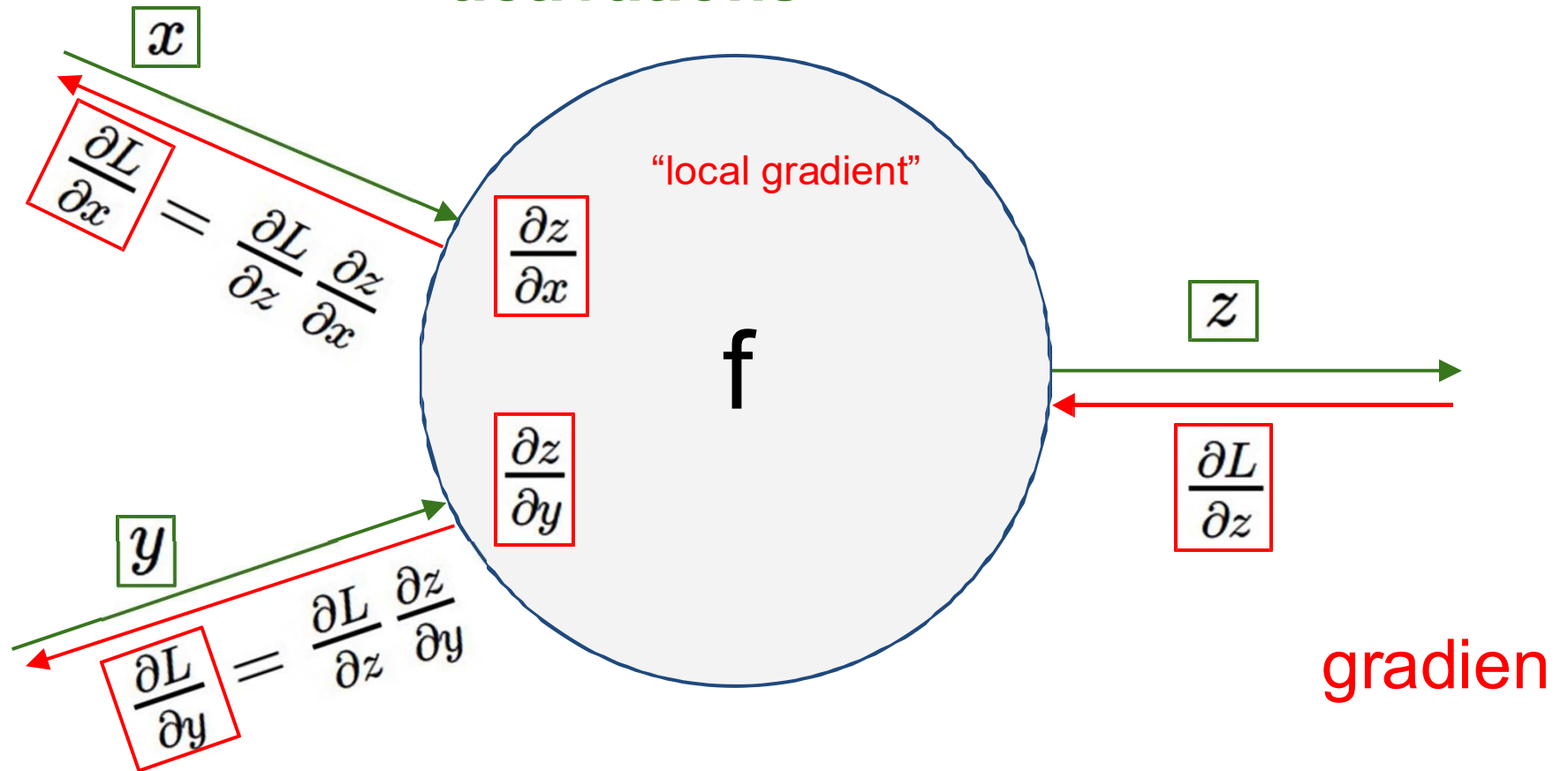




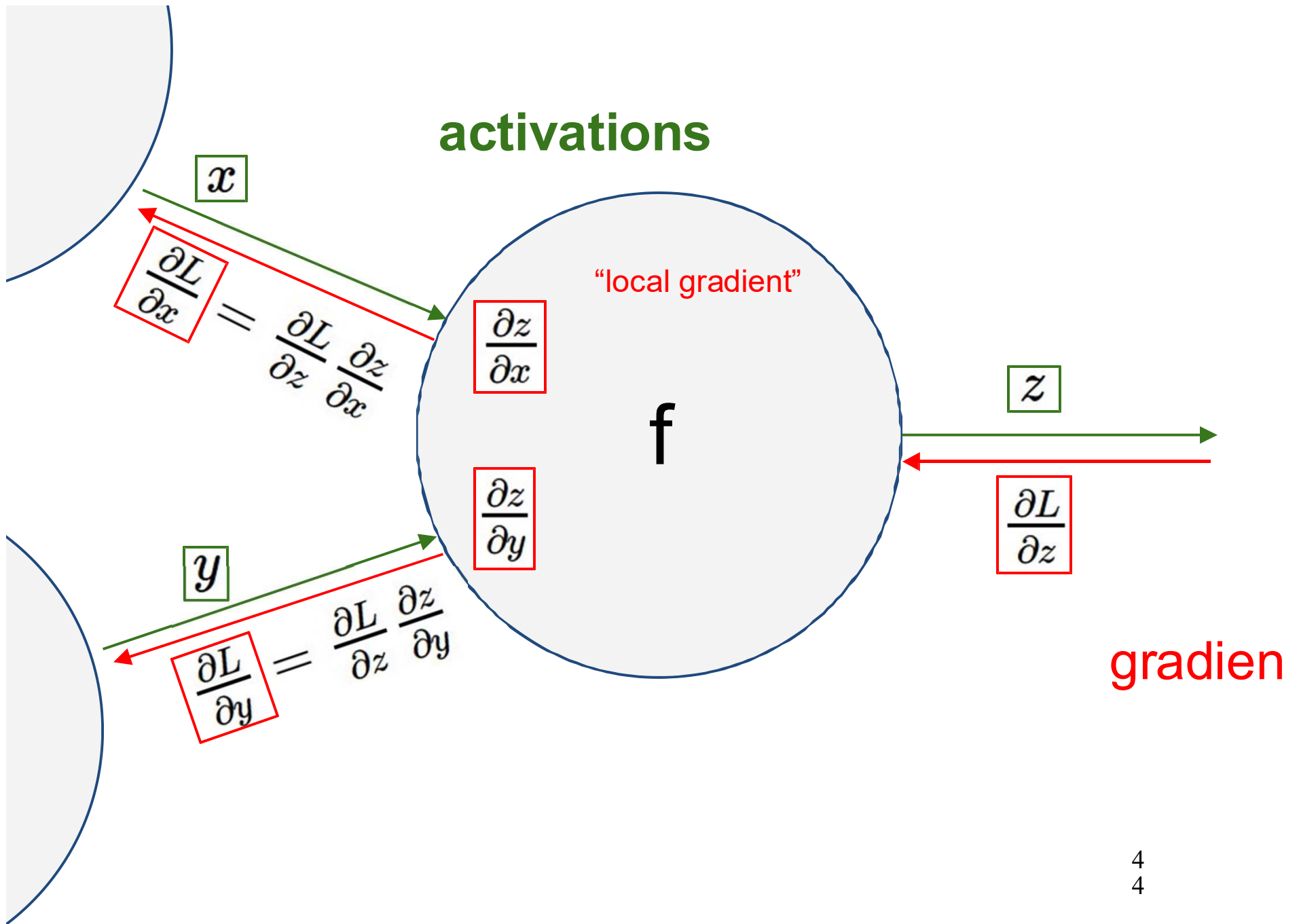
activations



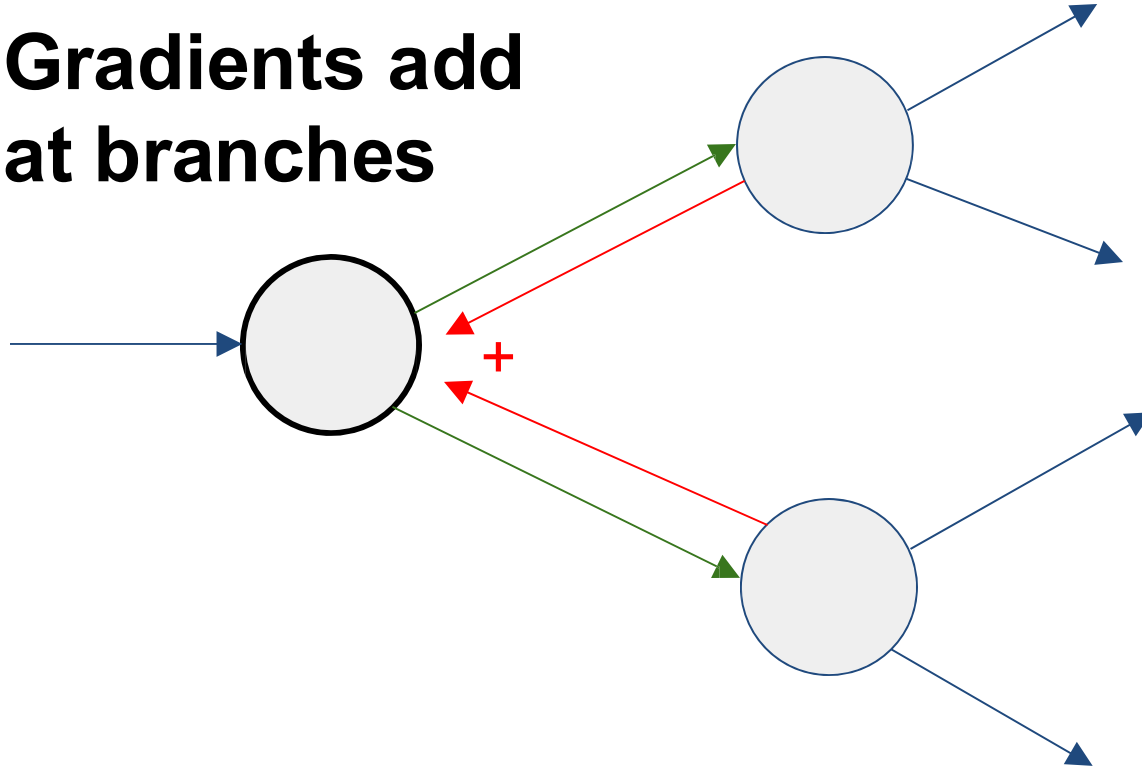
activations



activations



**Gradients add
at branches**



Backprop Summary

- Modern neural nets are very large
 - No hope of writing down gradient formula by hand for all parameters
- **Backpropagation**: recursive application of the chain rule along a computational graph to compute the gradients of all inputs/parameters/intermediates
- Should maintain a graph structure, where the nodes implement **forward()** and **backward()**
 - **forward**: compute result of an operation and save any intermediates needed for gradient computation in memory
 - **backward**: apply the chain rule to compute the gradient of the loss function with respect to the inputs

How does this work in practice?

1. Initialize all weights to small, non-zero values
2. Choose a training point (or mini-batch) and **clamp** to input layer
3. Propagate signal **forward** thru network until output is generated
4. Calculate deltas for output layer

Backprop. in practice (cont'd)

5. Calculate deltas for previous layers by propagating errors backwards until a delta is calculated for every unit
6. Calculate weight updates for every weight
7. Update weights (using some update strategy)
8. Goto step 2

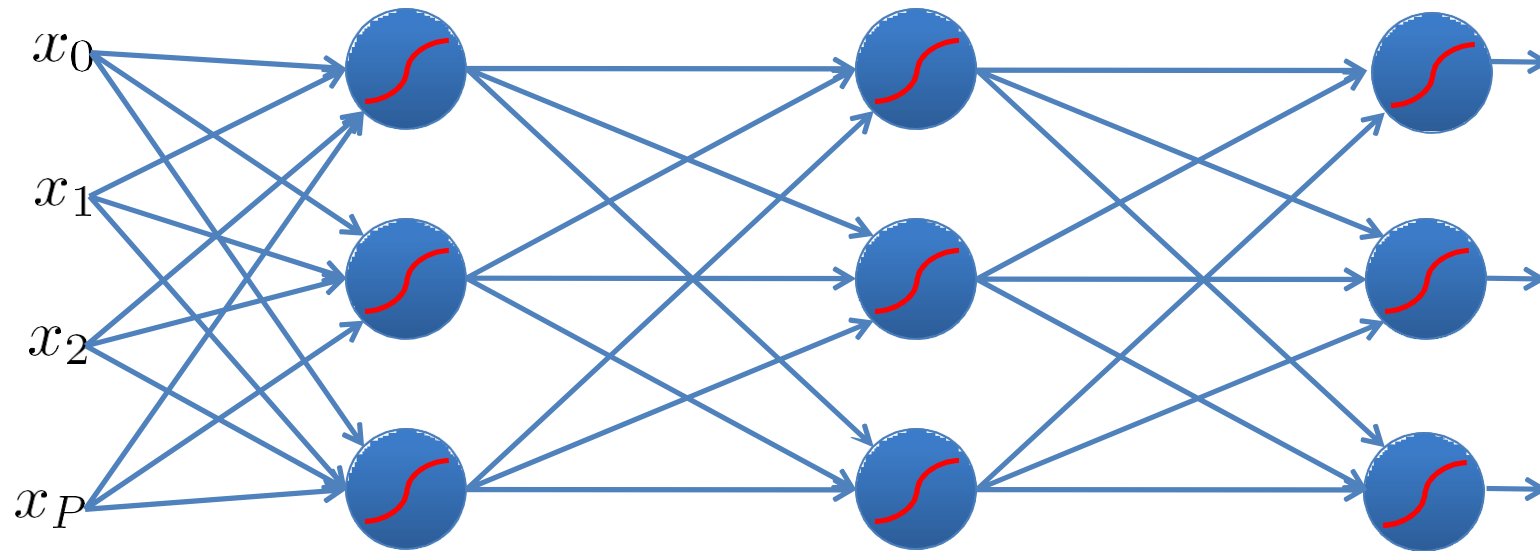
Notes on Backpropagation

- Learning is incremental
 - Training examples will be presented multiple times
 - Especially with small learning rates
- This is very basic backprop.
 - Other error functions
 - Momentum
 - Different learning rules
- Backprop. will only find local minima
- In practice, it works better with:
 - inputs in the range $[-1, +1]$
 - small weights

Other Neural Networks

- Backprop generalizes to networks that skip layers
- Many variants of ANN
 - Multiple Outputs
 - Skip Layer Network
 - Recurrent Neural Networks

Multiple Outputs



Used for N-way classification
Each output node corresponds to a different class
Training labels are $\langle 0, 0, 1, 0, 0, 0 \rangle$
Testing output: highest value wins!

Artificial Neural Networks

- MLPs are a network of simple units
 - Lead to complex learners
- Error backpropagation trains an MLP
 - Gradient descent applied to entire network
 - Trains network “one layer at a time”
- These types of MLPs are usually ≤ 3 layers
 - Vanishing gradients
 - Too many params

Introduction to Keras

– what is it?

An unified high level API (by Chollet) to describe, train, and evaluate deep neural nets. Can work with several computational backends.

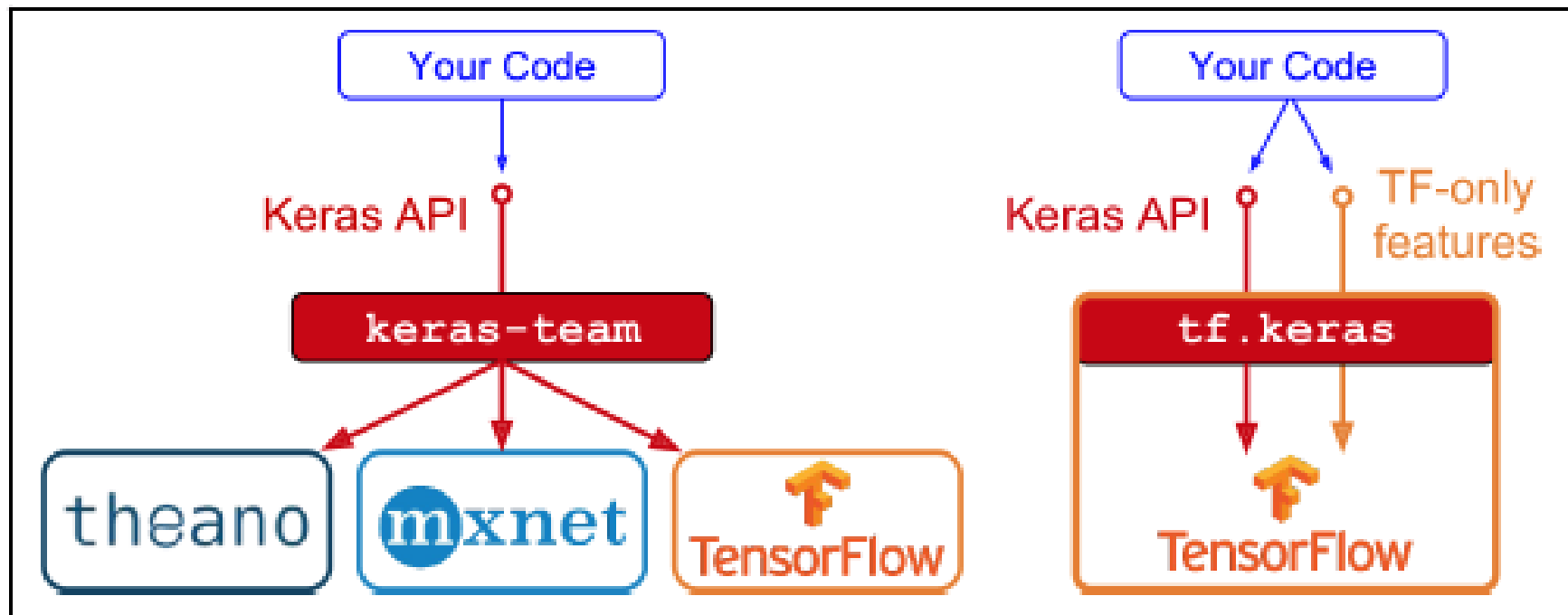


Figure 10-10. Two Keras implementations: *keras-team* (left) and *tf.keras* (right)

Your First MLP in Keras on TensorFlow

step 1: import the library

```
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt

tf.__version__
keras.__version__
```

step 2: import a toy dataset

```
fashion_mnist = keras.datasets.fashion_mnist
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist.load_data()
```

step 3: normalize and scale the toy dataset

```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255.
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]
X_test = X_test / 255.

class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat", "Sandal",
               "Shirt", "Sneaker", "Bag", "Ankle boot"]
```

example image

```
class_names[y_train[0]]  
plt.imshow(X_train[0], cmap="binary")  
plt.axis('off')  
plt.show()
```



Your First MLP in Keras on TensorFlow

more example images

```
n_rows = 4
n_cols = 10
plt.figure(figsize=(n_cols * 1.2, n_rows * 1.2))
for row in range(n_rows):
    for col in range(n_cols):
        index =
            n_cols * row + col
        plt.subplot(n_rows, n_cols, index + 1)
        plt.imshow(X_train[index], cmap="binary", interpolation="nearest")
        plt.axis('off')
        plt.title(class_names[y_train[index]], fontsize=12)
plt.subplots_adjust(wspace=0.2, hspace=0.5)
plt.show()
```




Your First MLP in Keras on TensorFlow

step 4: describe the shape of the NN

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
    keras.layers.Dense(10, activation="softmax")
])
```

print and describe model shape with the following

```
model.summary()
keras.utils.plot_model(model, "my_fashion_mnist_model.png",
    show_shapes=True)
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 300)	235,500
dense_1 (Dense)	(None, 100)	30,100
dense_2 (Dense)	(None, 10)	1,010

Total params: 266,610 (1.02 MB)

Trainable params: 266,610 (1.02 MB)

Non-trainable params: 0 (0.00 B)

Your First MLP in Keras on TensorFlow

step 5: tell it which optimizer, the loss to minimize, and the metrics to measure

```
model.compile(loss="sparse_categorical_crossentropy",  
              optimizer="sgd",  
              metrics=["accuracy"])
```

step 6: fit the model, keeping track of validation loss

```
history = model.fit(X_train, y_train, epochs=30,  
                    validation_data=(X_valid, y_valid))
```

step 7: plot learning curves

```
import pandas as pd  
pd.DataFrame(history.history).plot(figsize=(8, 5))  
plt.grid(True)  
plt.gca().set_ylim(0, 1)  
plt.show()
```

step 8: evaluate the trained model on testing data

```
model.evaluate(X_test, y_test)
```

Epoch 1/30
 1719/1719 ————— 2s 810us/step - accuracy: 0.6847 - loss: 1.0173 - val_accuracy: 0.8162 - val_loss: 0.5361

Epoch 2/30
 1719/1719 ————— 1s 759us/step - accuracy: 0.8235 - loss: 0.5093 - val_accuracy: 0.8414 - val_loss: 0.4694

Epoch 3/30
 1719/1719 ————— 1s 770us/step - accuracy: 0.8398 - loss: 0.4583 - val_accuracy: 0.8432 - val_loss: 0.4680

Epoch 4/30
 1719/1719 ————— 1s 767us/step - accuracy: 0.8525 - loss: 0.4198 - val_accuracy: 0.8600 - val_loss: 0.4103

Epoch 5/30
 1719/1719 ————— 1s 754us/step - accuracy: 0.8583 - loss: 0.4045 - val_accuracy: 0.8566 - val_loss: 0.4093

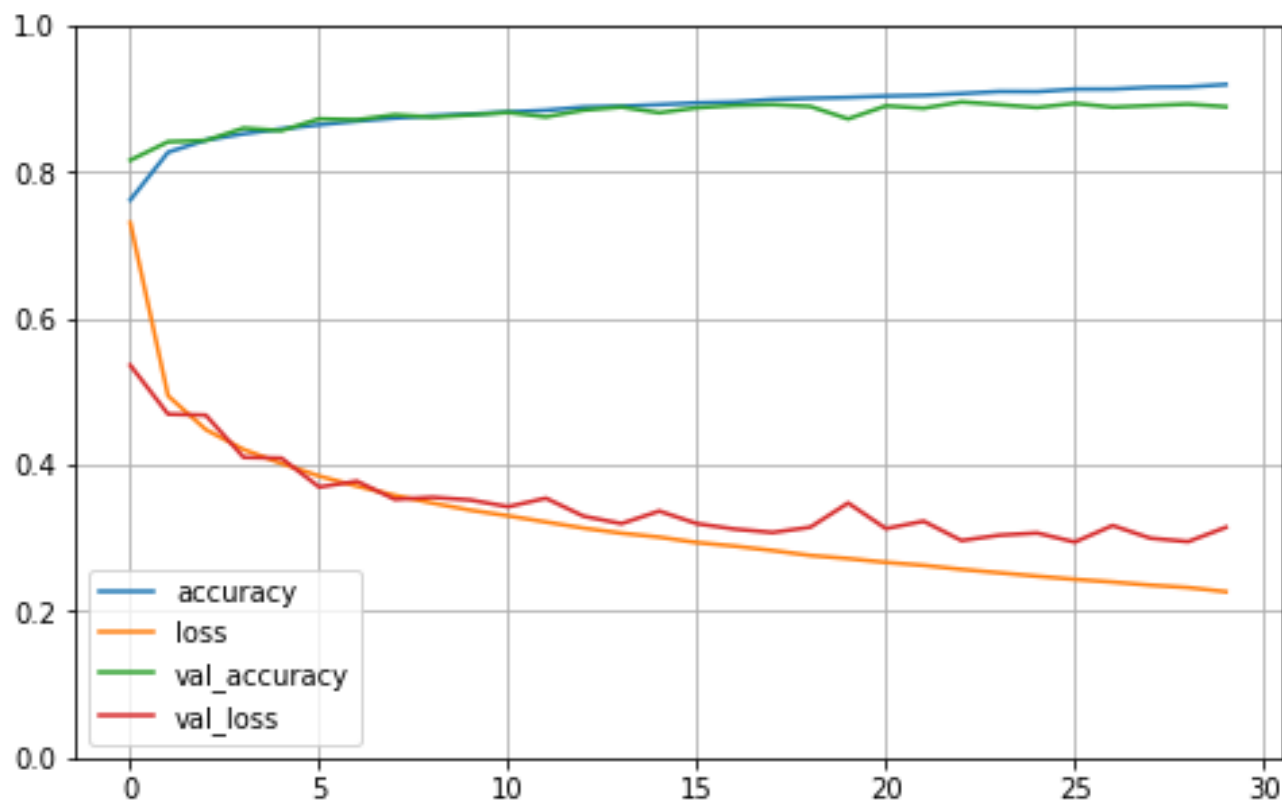
Epoch 6/30
 1719/1719 ————— 1s 755us/step - accuracy: 0.8641 - loss: 0.3854 - val_accuracy: 0.8726 - val_loss: 0.3698

Epoch 7/30
 1719/1719 ————— 1s 762us/step - accuracy: 0.8704 - loss: 0.3680 - val_accuracy: 0.8714 - val_loss: 0.3777

Epoch 8/30
 1719/1719 ————— 1s 739us/step - accuracy: 0.8718 - loss: 0.3600 - val_accuracy: 0.8782 - val_loss: 0.3530

Epoch 9/30
 1719/1719 ————— 1s 746us/step - accuracy: 0.8755 - loss: 0.3498 - val_accuracy: 0.8750 - val_loss: 0.3558

accuracy: 0.8836 - loss: 0.3377



Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Import Dataset

Import a toy dataset

```
fashion_mnist = keras.datasets.fashion_mnist  
(X_train_full, y_train_full), (X_test, y_test) = fashion_mnist.load_data()
```

MNIST (70,000 grayscale images
of 28×28 pixels each, with
10 classes)



normalize and scale the toy dataset

```
X_valid, X_train = X_train_full[:5000] / 255., X_train_full[5000:] / 255.  
y_valid, y_train = y_train_full[:5000], y_train_full[5000:]  
X_test = X_test / 255.
```



Figure 10-11. Samples from Fashion MNIST

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Artificial Neural Networks & Multi-Layer Perceptions – Activation Functions

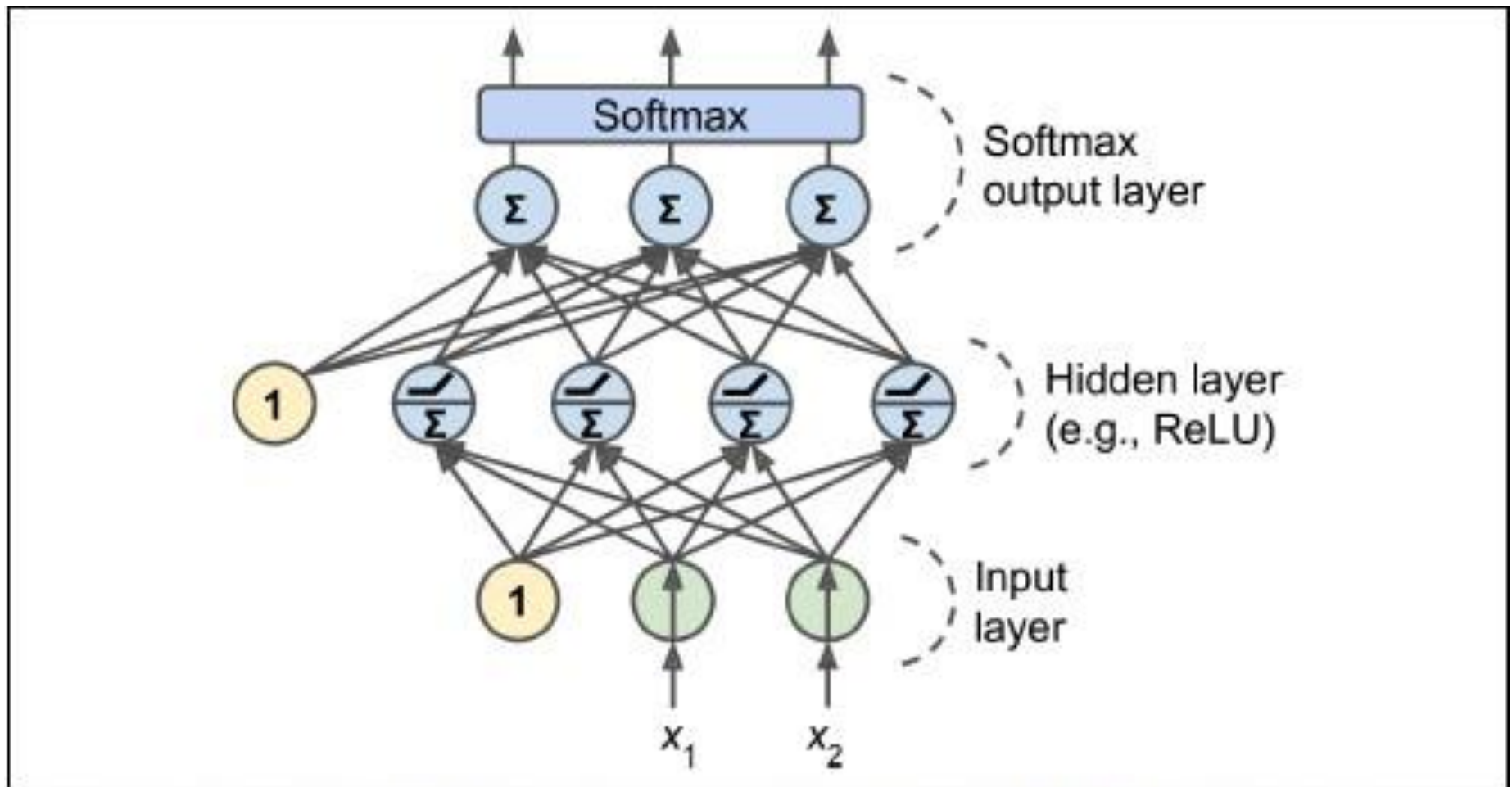


Figure 10-9. A modern MLP (including ReLU and softmax) for classification

Model Creation Options

- `keras.models.Sequential([*])`

- argument is a list of layers,

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300,activation="relu"),
    keras.layers.Dense(100,activation="relu"),
    keras.layers.Dense(1)])
```

- or can use

```
model = keras.models.Sequential()
model.add(keras.layers.InputLayer(input_shape=input_shape))
for layer in range(n_hidden):
    model.add(keras.layers.Dense(n_neurons, activation="relu"))
model.add(keras.layers.Dense(1))
```

- layers are stacked on top of one another

Model Creation Options

- most common
- `keras.Model`
 - Interconnect layers when defining them & create by specifying input and output

```
base_model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dense(300, activation="relu"),
    keras.layers.Dense(100, activation="relu"),
])
output=keras.layers.Dense(10,activation="softmax")(base_model.output)
model=keras.models.Model(inputs=base_model.input,outputs=output)
```

when you need more flexibility and control over your model architecture and training process, and prefer `keras.Sequential` for simpler, linear stack models where ease of use and rapid prototyping are more important.

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Layers Options - Overview

- dense layers
- activation function options
- convolutional layers
- pooling (avg,max)
- regularization w/ dropout
- more coming: concatenate, recurrent layers, attention layers

Layers - Dense

```
keras.layers.Dense(10)
```

- Parameter is number of output neurons.
- Each output neuron connected to all inputs with different weight and bias.
- $\text{num weights} = (\text{num neurons}) \times (\text{num inputs} + 1)$

```
>>> model.summary()
```

Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 300)	235500
dense_4 (Dense)	(None, 100)	30100
dense_5 (Dense)	(None, 10)	1010
Total params: 266,610		
Trainable params: 266,610		
Non-trainable params: 0		

Layers - Dense

```
keras.layers.Dense(10)
```

- Parameter is number of output neurons.
- Each output neuron connected to all inputs with different weight and bias.
- can specify activation function

```
keras.layers.Dense(10, activation="XX")
```

where XX selected from

relu, selu, softmax, leaky_relu, sigmoid, tanh

, others.

Default (activation omitted) is no nonlinearity.

Layers - Activation Function Options

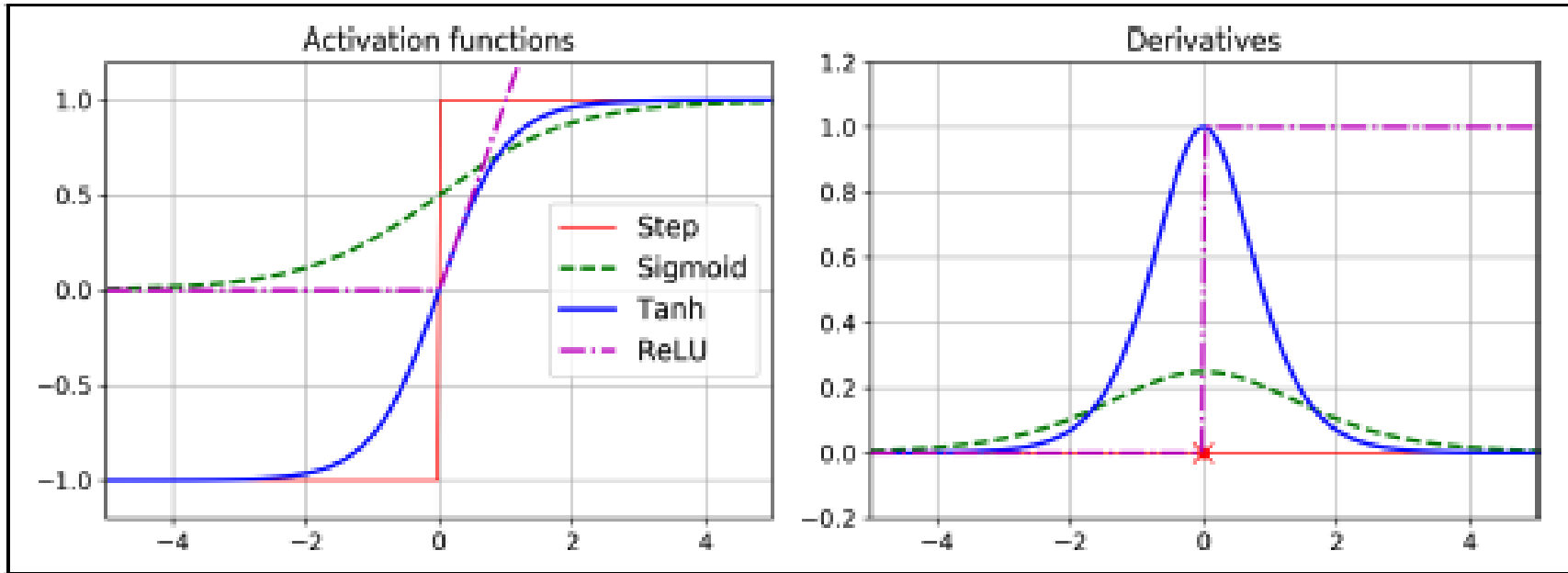


Figure 10-8. Activation functions and their derivatives

Equation 4-20. Softmax function

$$\hat{p}_k = \sigma(\mathbf{s}(\mathbf{x}))_k = \frac{\exp(s_k(\mathbf{x}))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}))}$$

- K is the number of classes.
- $\mathbf{s}(\mathbf{x})$ is a vector containing the scores of each class for the instance $\mathbf{x}$.
- $\sigma(\mathbf{s}(\mathbf{x}))_k$ is the estimated probability that the instance $\mathbf{x}$ belongs to class k given the scores of each class for that instance.

Layers - Activation Function Options - ReLU

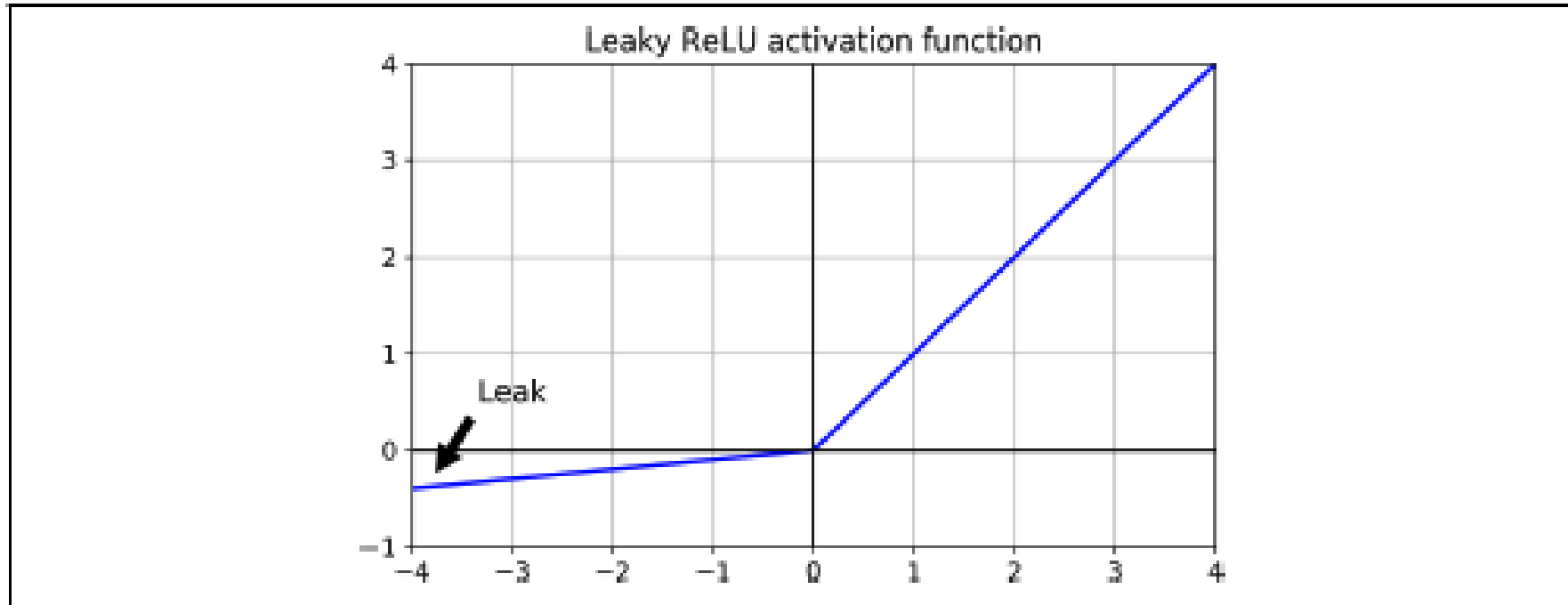


Figure 11-2. Leaky ReLU

```
leaky_relu = keras.layers.LeakyReLU(alpha=0.2)
layer = keras.layers.Dense(10, activation=leaky_relu,
                             kernel_initializer="he_normal")
```

$$\text{ReLU}_{\alpha}(z) = \max(\alpha z, z)$$

Layers - Activation Function Options - SeLU

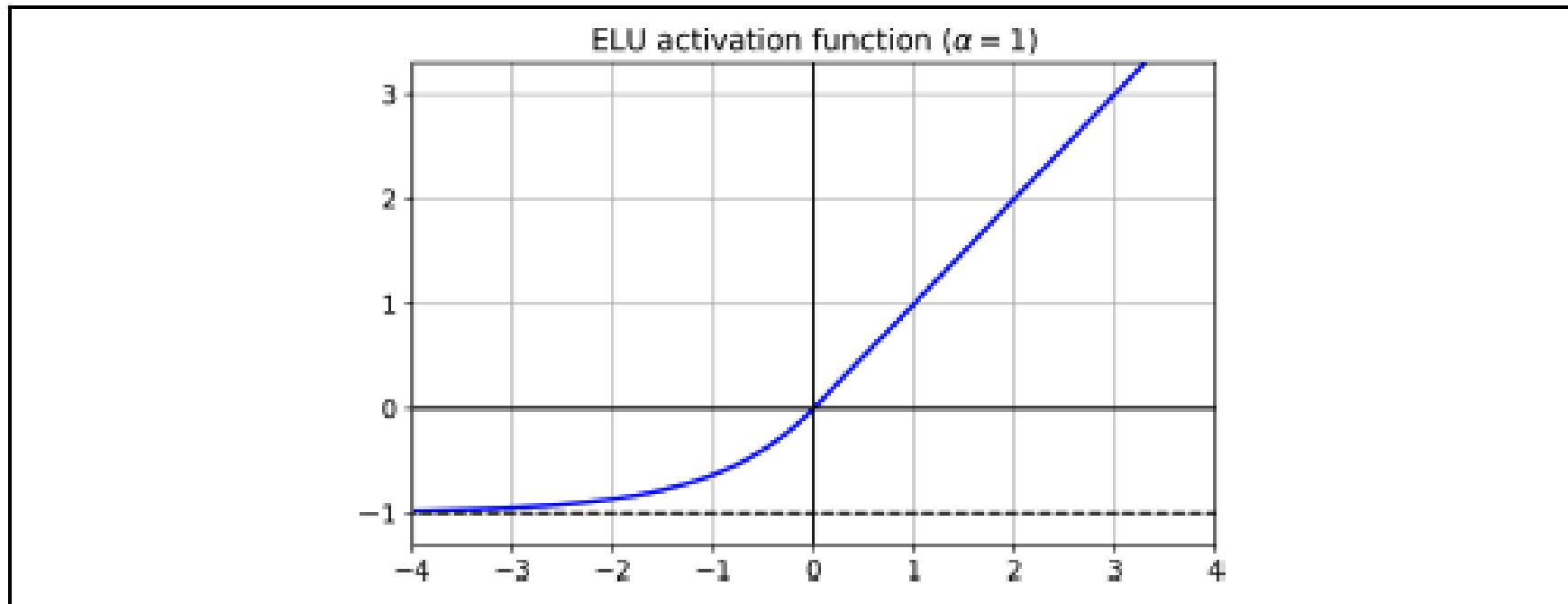


Figure 11-3. ELU activation function

```
layer = keras.layers.Dense(10, activation="selu",  
                             kernel_initializer="lecun_normal")
```

The Scaled Exponential Linear Unit (SELU) activation function is defined as:

scale * x if $x > 0$

scale * alpha * (exp(x) - 1) if $x < 0$

where alpha and scale are pre-defined constants

(alpha=1.67326324 and scale=1.05070098).

Proper initialization is an active area of research

Understanding the difficulty of training deep feedforward neural networks by Glorot and Bengio, 2010

Exact solutions to the nonlinear dynamics of learning in deep linear neural networks by Saxe et al, 2013

Random walk initialization for training very deep feedforward networks by Sussillo and Abbott, 2014

Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification by He et al., 2015

Data-dependent Initializations of Convolutional Neural Networks by Krähenbühl et al., 2015

All you need is a good init, Mishkin and Matas, 2015

Fixup Initialization: Residual Learning Without Normalization, Zhang et al, 2019

The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks, Frankle and Carbin, 2019

Weight Initialization in Neural Networks

Importance of Weight Initialization:

- Proper weight initialization is crucial for the effective training of deep neural networks. It helps in avoiding issues like vanishing or exploding gradients.

Common Weight Initialization Methods:

1. Zero Initialization

2. Random Initialization:

- Weights are initialized with small random values.
- Simple but can lead to slow convergence or poor performance.

3. Xavier/Glorot Initialization:

- Weights are drawn from a distribution with mean 0 and variance $2/(n_{in} + n_{out})$
- Usage: Suitable for layers with sigmoid or tanh activation functions.

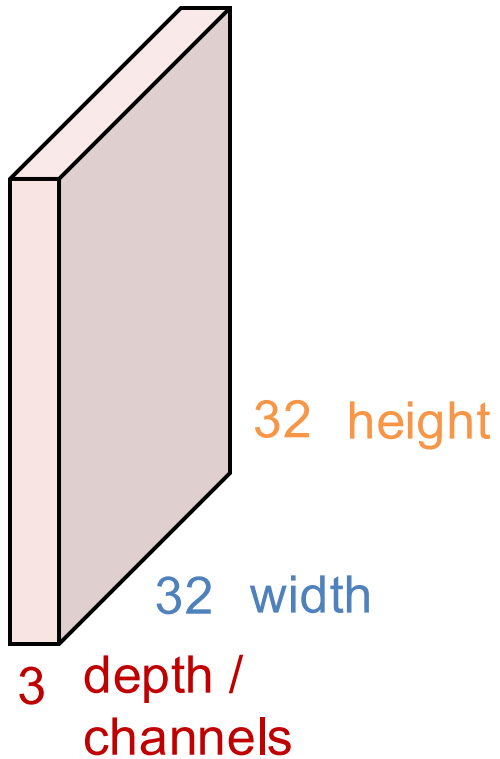
4. He Initialization:

- Weights are drawn from a distribution with mean 0 and variance $2/n_{in}$
- Optimal for layers with ReLU activation functions.

```
layer = keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal")
```

Layers - Convolutional

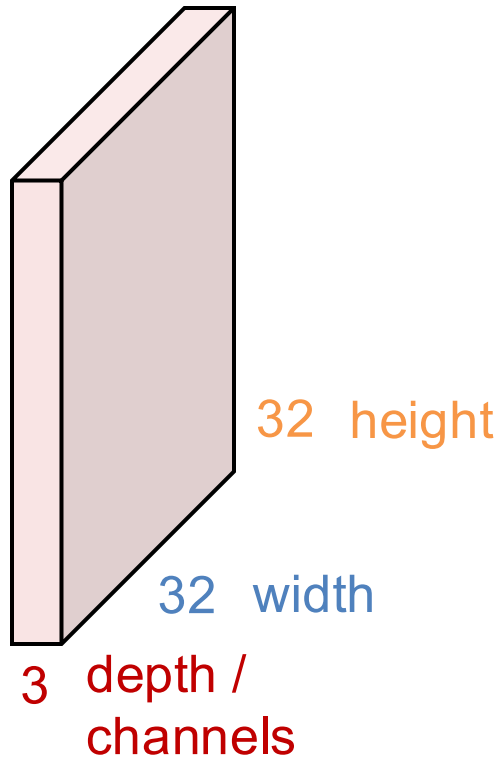
3x32x32 image



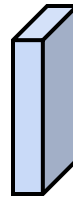
Convolution Layer: preserve spatial structure,
Share the weights

Layers - Convolutional

3x32x32 image

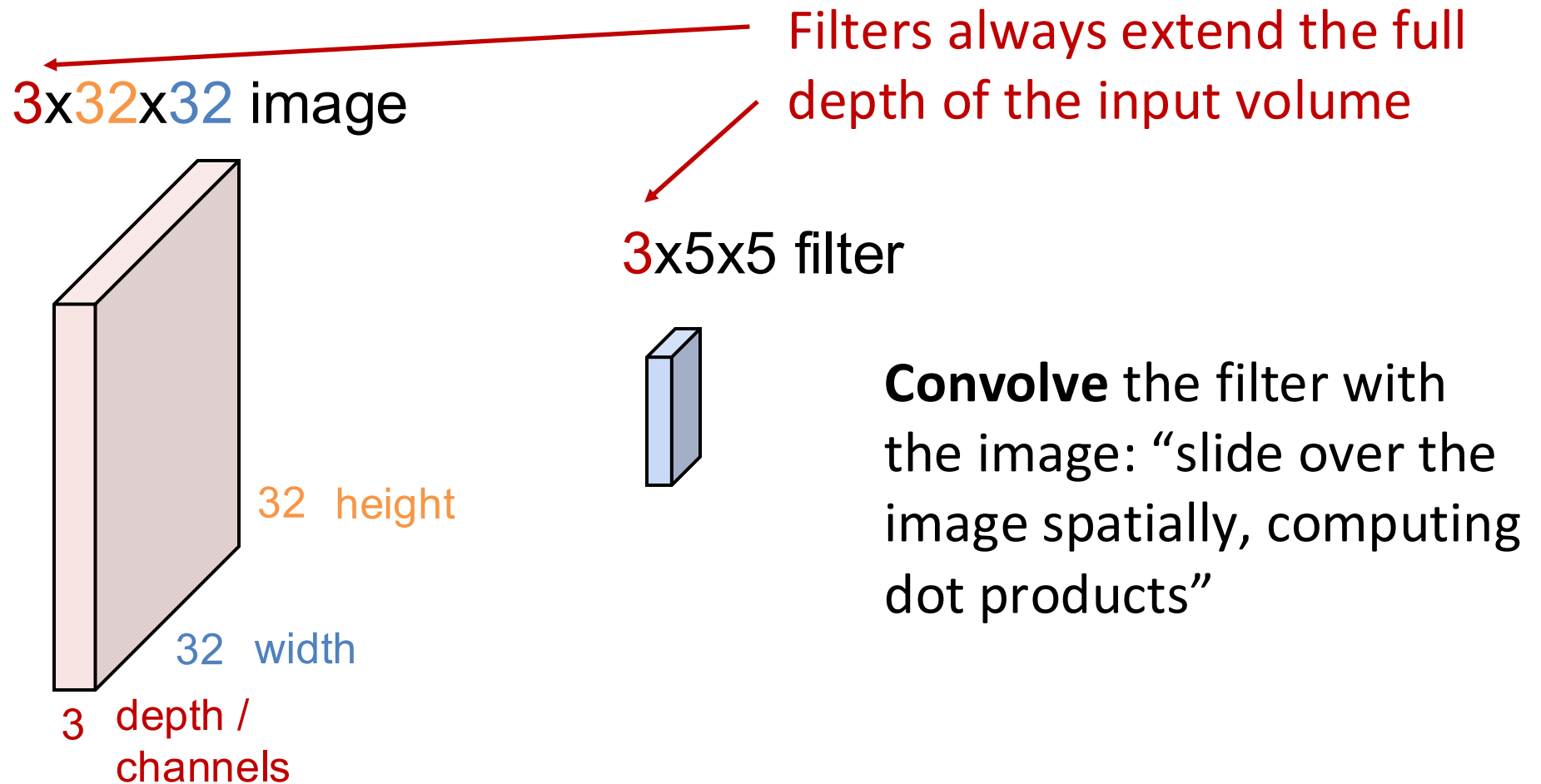


3x5x5 filter



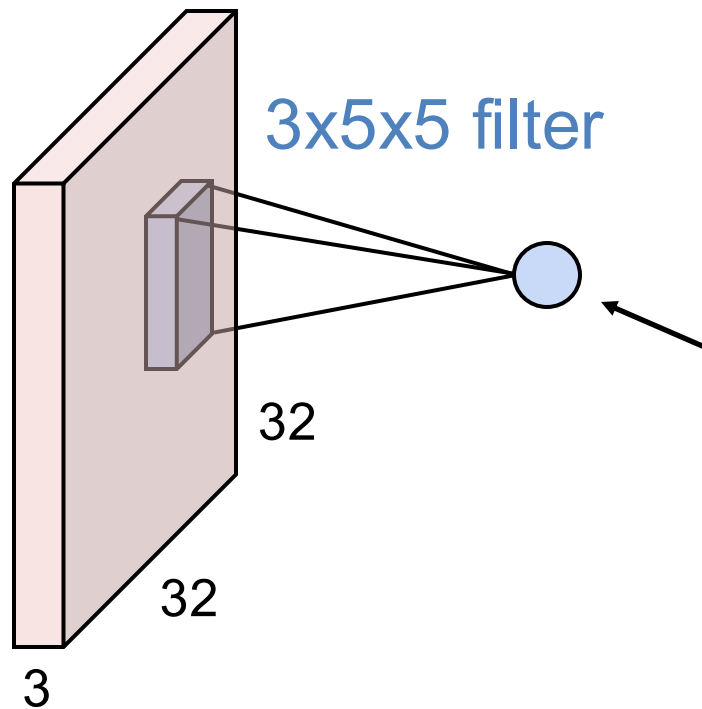
Convolve the filter with the image: “slide over the image spatially, computing dot products”

Layers - Convolutional



Layers - Convolutional

3x32x32 image



1 number:

the result of taking a dot product between the filter and a small 3x5x5 chunk of the image (i.e. $3 \times 5 \times 5 = 75$ -dimensional dot product + bias)

$$w^T x + b$$

Convolutional Layer, *Receptive Fields*

- The receptive field of a neuron in a CNN refers to the specific region of the input image that influences the neuron's activation. It is the area of the input that the neuron's filters are "looking at" when determining their output.

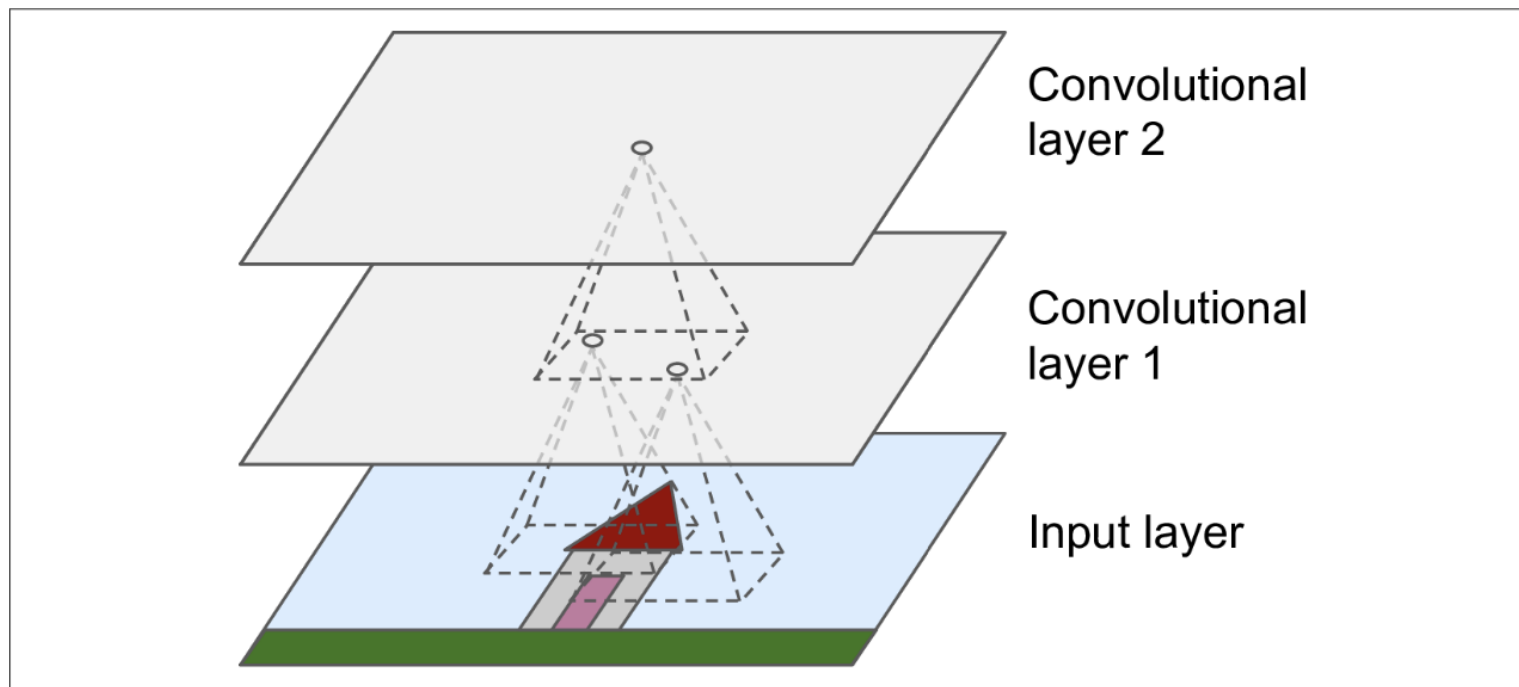
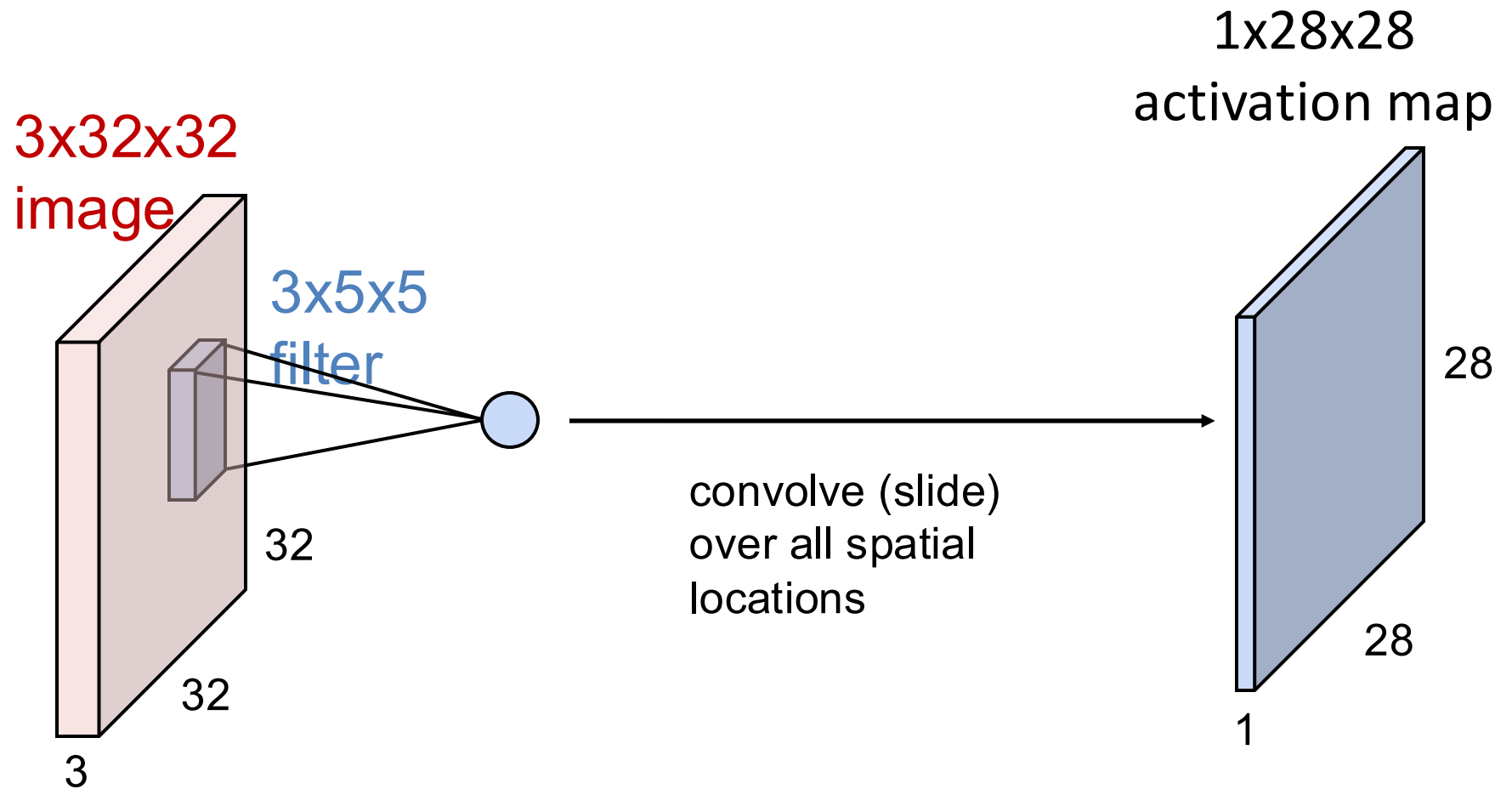


Figure 14-2. CNN layers with rectangular local receptive fields

Convolution Layer



Filter

Layers - Convolutional – What are they?

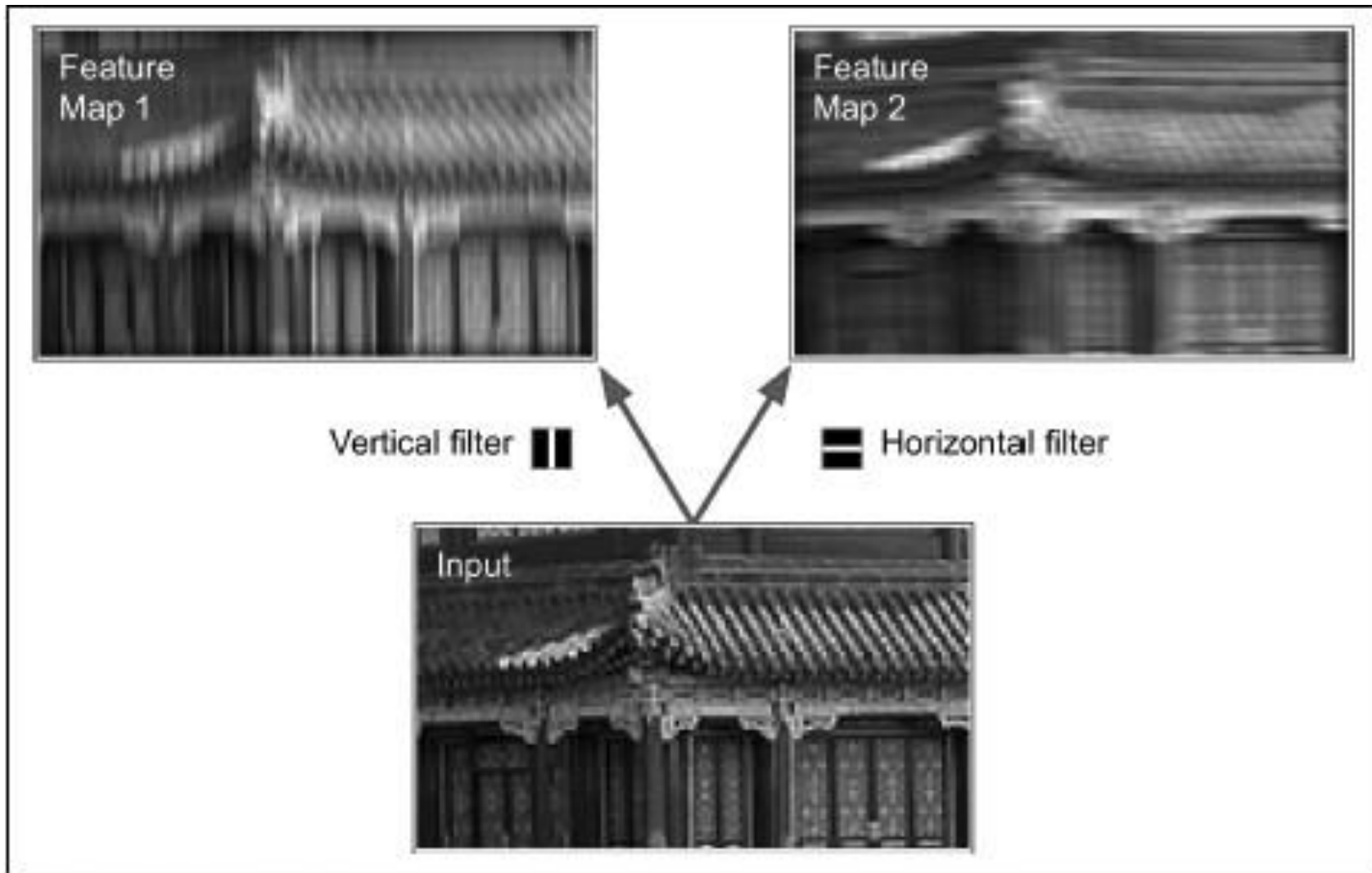
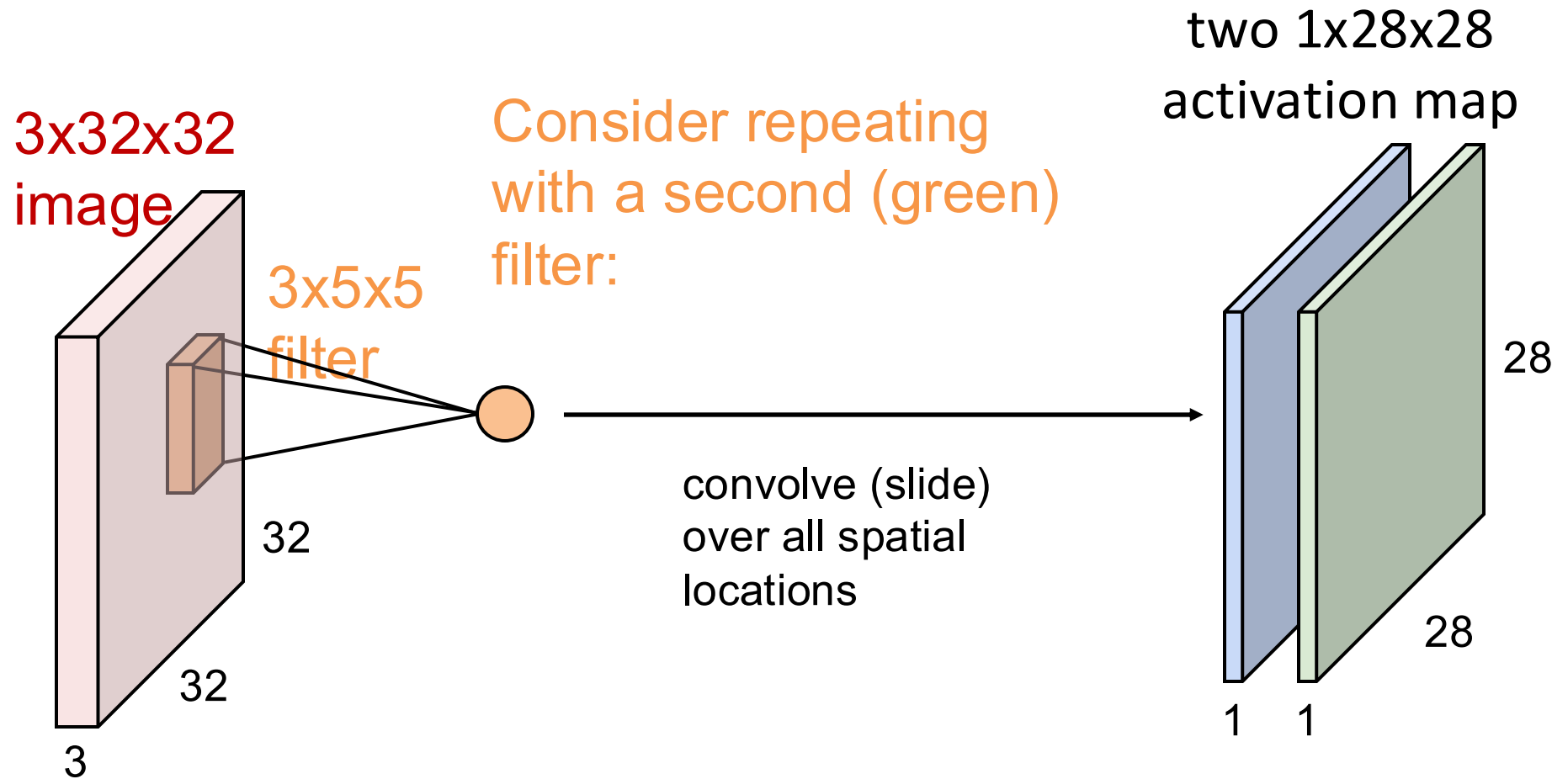


Figure 14-5. Applying two different filters to get two feature maps

Convolution Layer



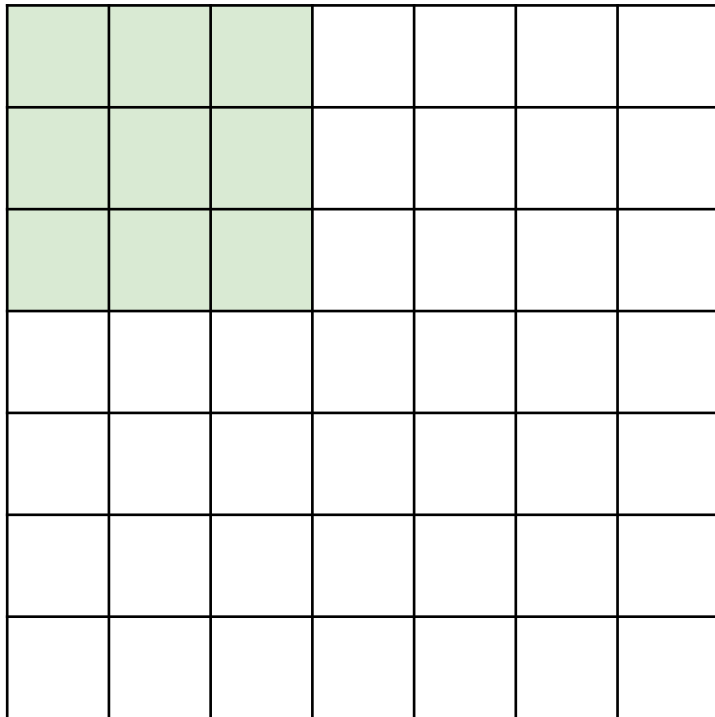
Convolution Spatial Dimensions

Input: 7x7

Filter: 3x3

Stride: 1

Q: How big is output?



7

7

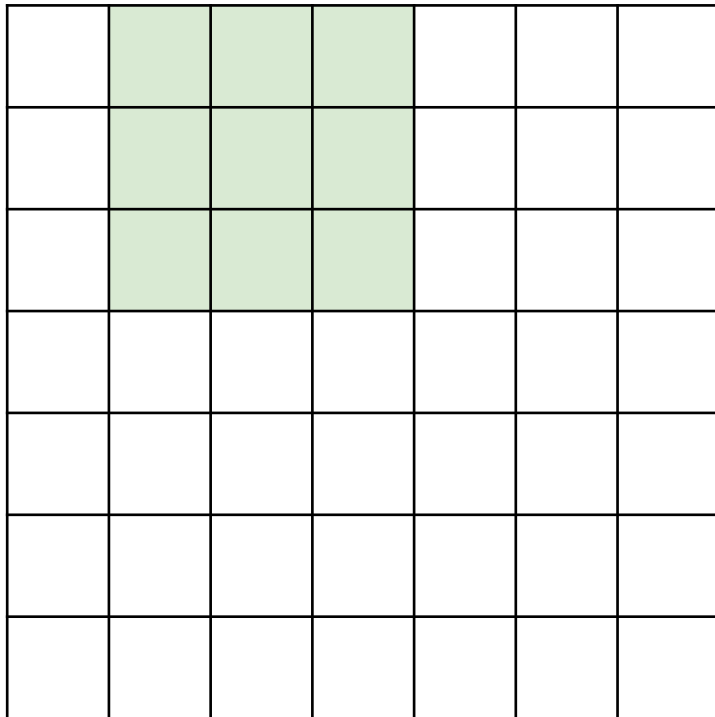
Convolution Spatial Dimensions

Input: 7x7

Filter: 3x3

Stride: 1

Q: How big is output?



7

7

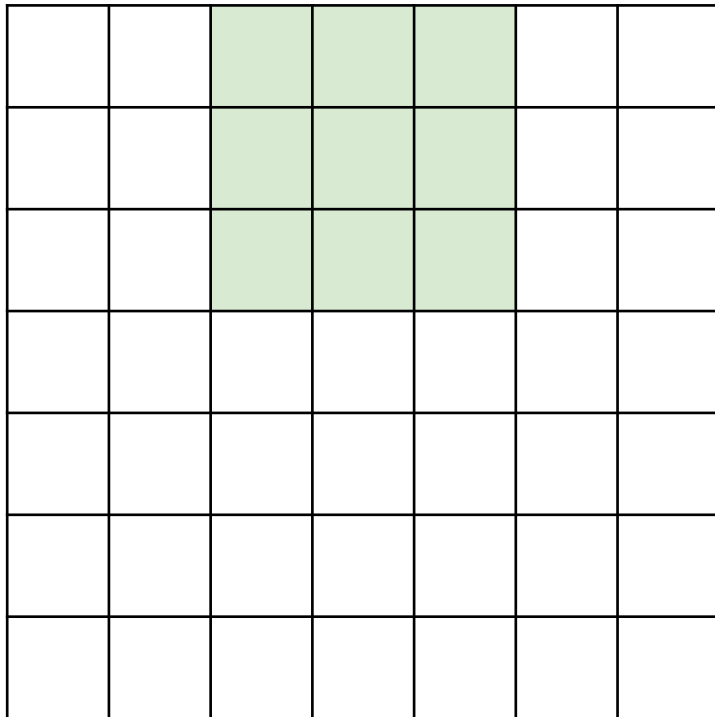
Convolution Spatial Dimensions

Input: 7x7

Filter: 3x3

Stride: 1

Q: How big is output?



7

7

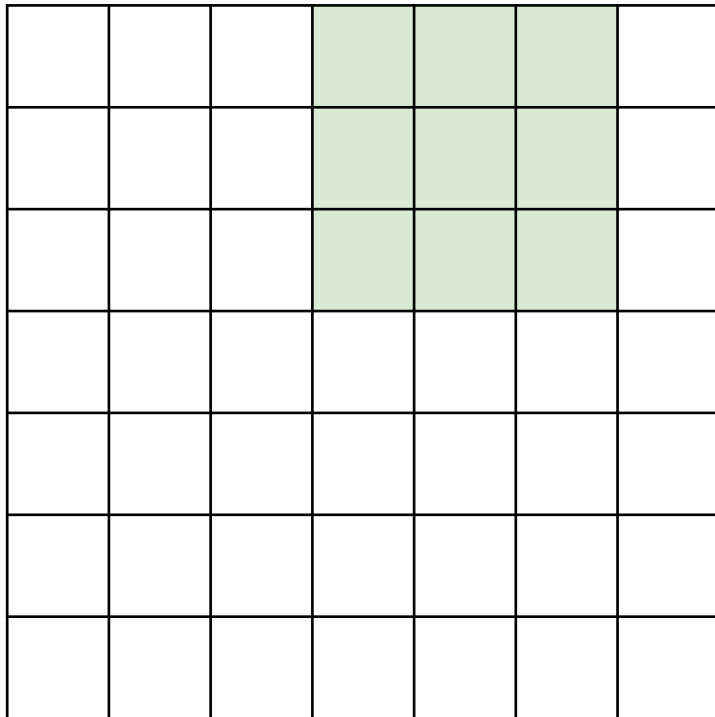
Convolution Spatial Dimensions

Input: 7x7

Filter: 3x3

Stride: 1

Q: How big is output?



7

7

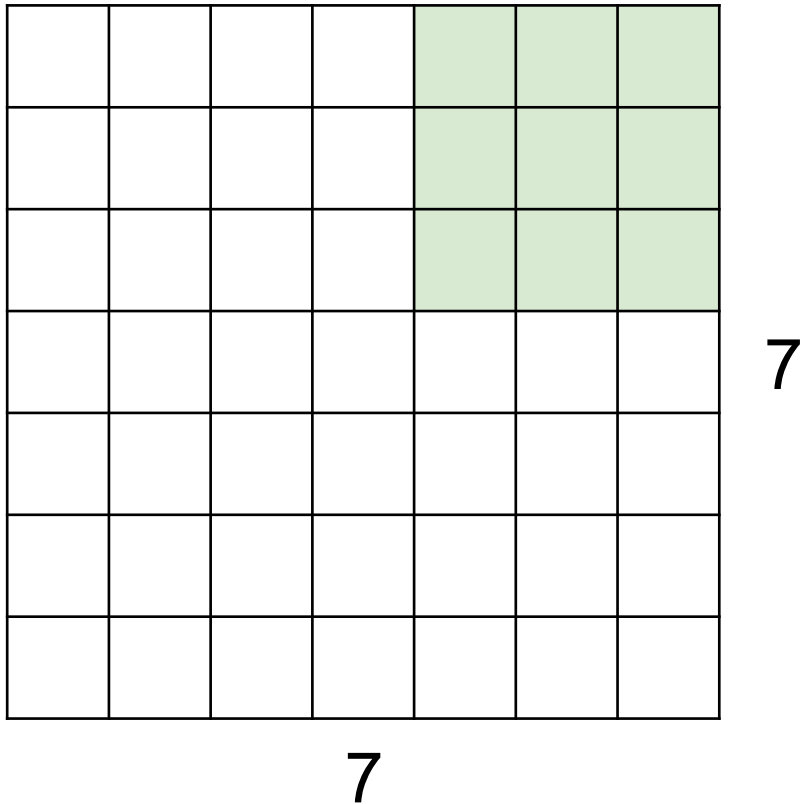
Convolution Spatial Dimensions

Input: 7x7

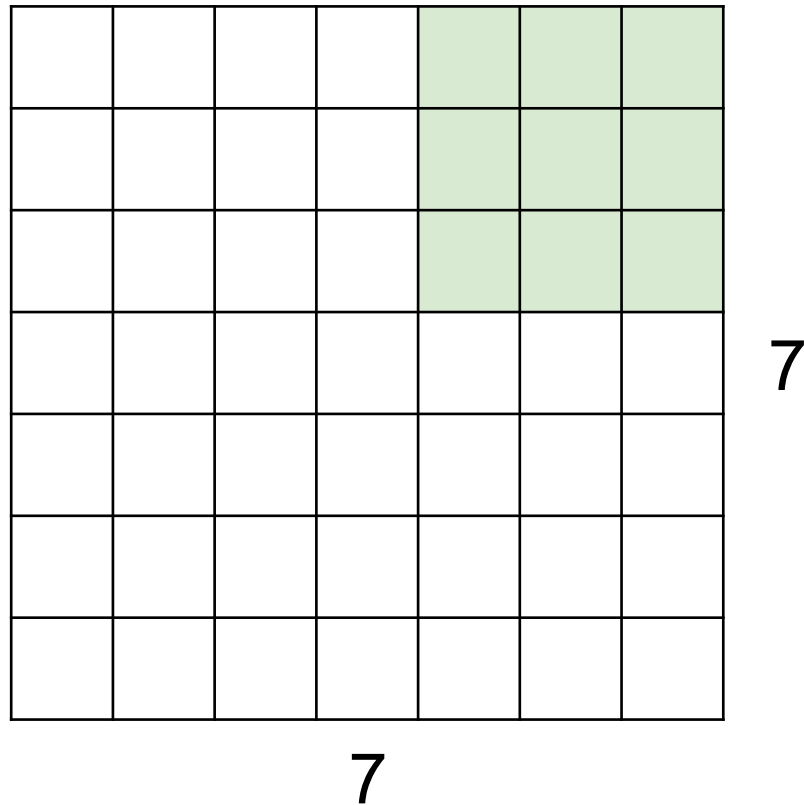
Filter: 3x3

Stride: 1

Output: 5x5



Convolution Spatial Dimensions



Input: 7x7

Filter: 3x3

Stride: 1

Output: 5x5

Problem:
Feature maps
“shrink” with
each layer!

In general:

Input: W

Filter: K

Stride: S

Output: $((W - K) / S) + 1$

Convolution Spatial Dimensions

0	0	0	0	0	0	0	0	0
0								0
0								0
0								0
0								0
0								0
0								0
0								0
0	0	0	0	0	0	0	0	0

Input: 7x7

Filter: 3x3

Stride: 2

Output: 5x5

In general:

Input: W

Filter: K

Stride: S

Padding: P

Problem:
Feature maps
“shrink” with
each layer!

Solution: padding

Very common: “same padding”,
Add zeros around the input

Set $P = (K - 1) / 2$

Then output size = input size

Strided Convolution

Input: 7x7

Filter: 3x3

Stride: 2

Output: 3x3

In general:

Input: W

Filter: K

Padding: P

Stride: S

Output: $(W - K + 2P) / S + 1$

Layers - Convolutional – What are they?

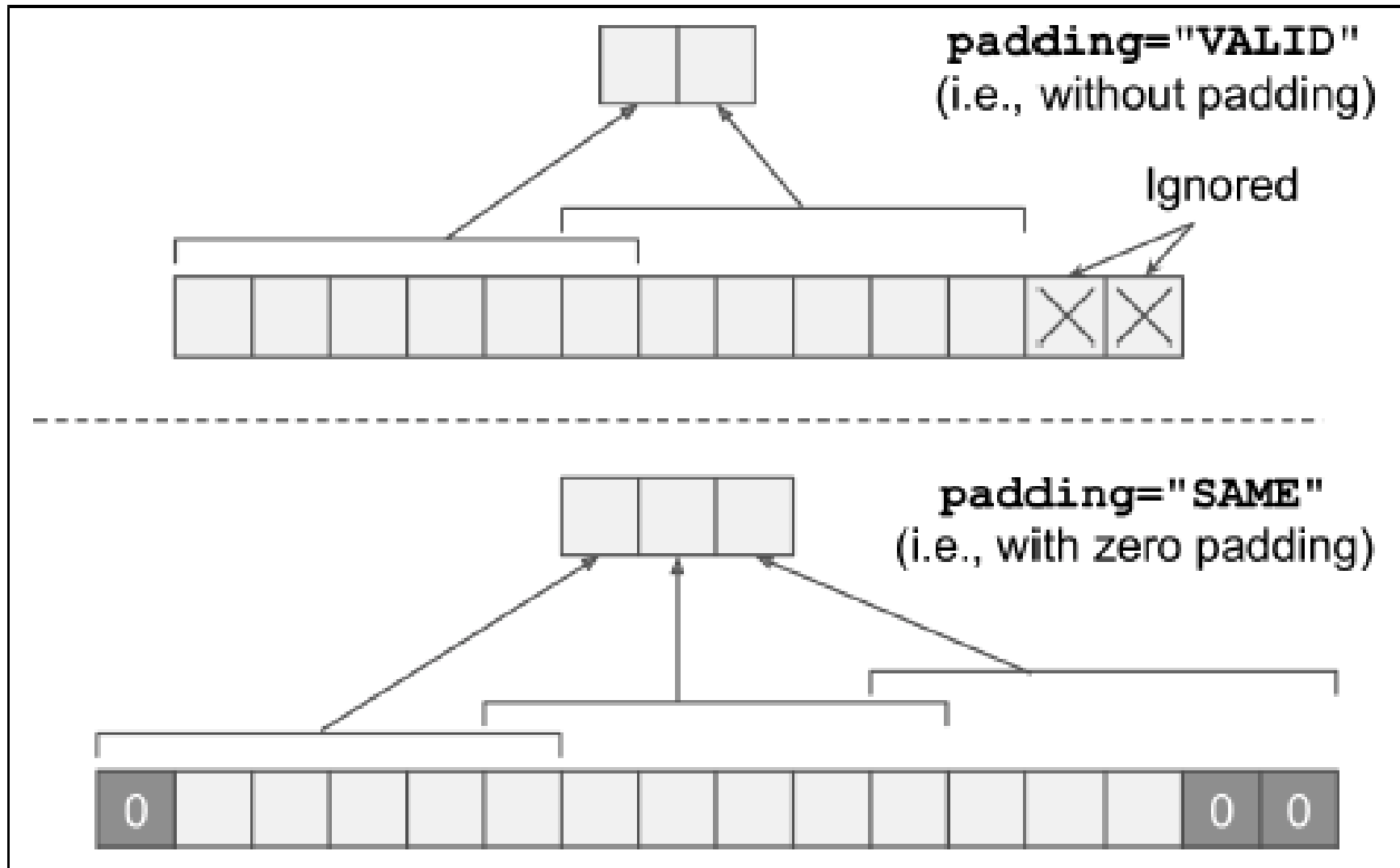


Figure 14-7. Padding options—input width: 13, filter width: 6, stride: 5

Layers - Convolutional

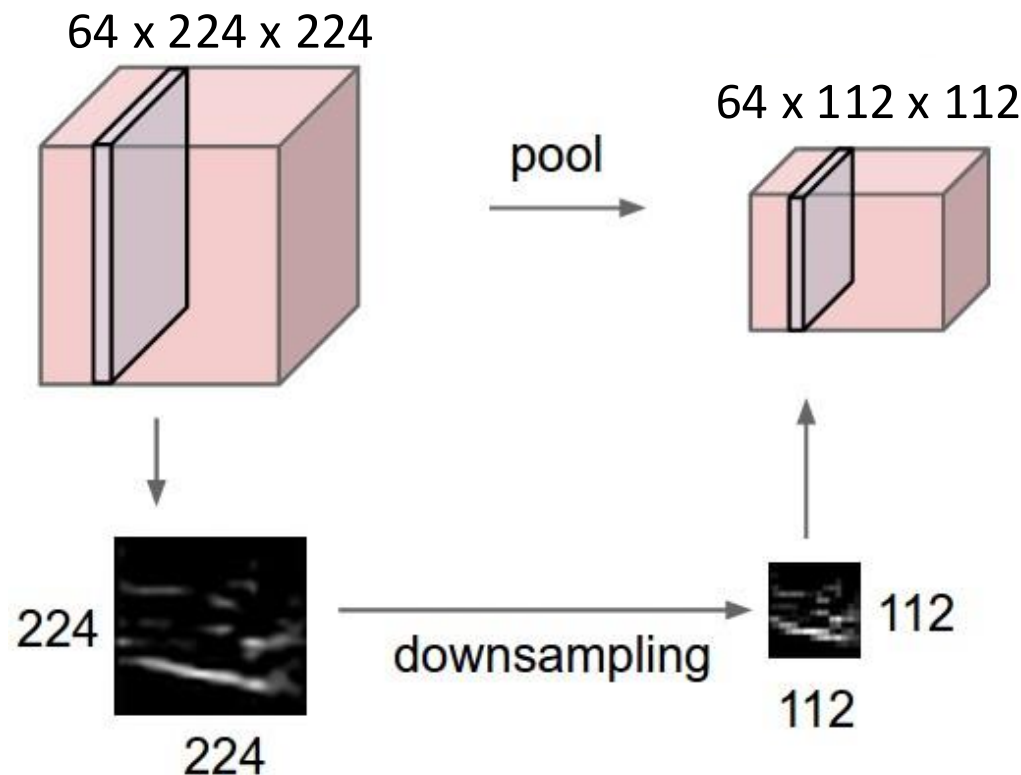
work best, as explained earlier. Instead of manually creating the variables, however, you can simply use the `keras.layers.Conv2D` layer:

```
conv = keras.layers.Conv2D(filters=32, kernel_size=3, strides=1,  
                             padding="SAME", activation="relu")
```

This code creates a Conv2D layer with 32 filters, each 3×3 , using a stride of 1 (both horizontally and vertically), SAME padding, and applying the ReLU activation function to its outputs. As you can see, convolutional layers have quite a few hyperpara-

Pooling Layers: Downsampling

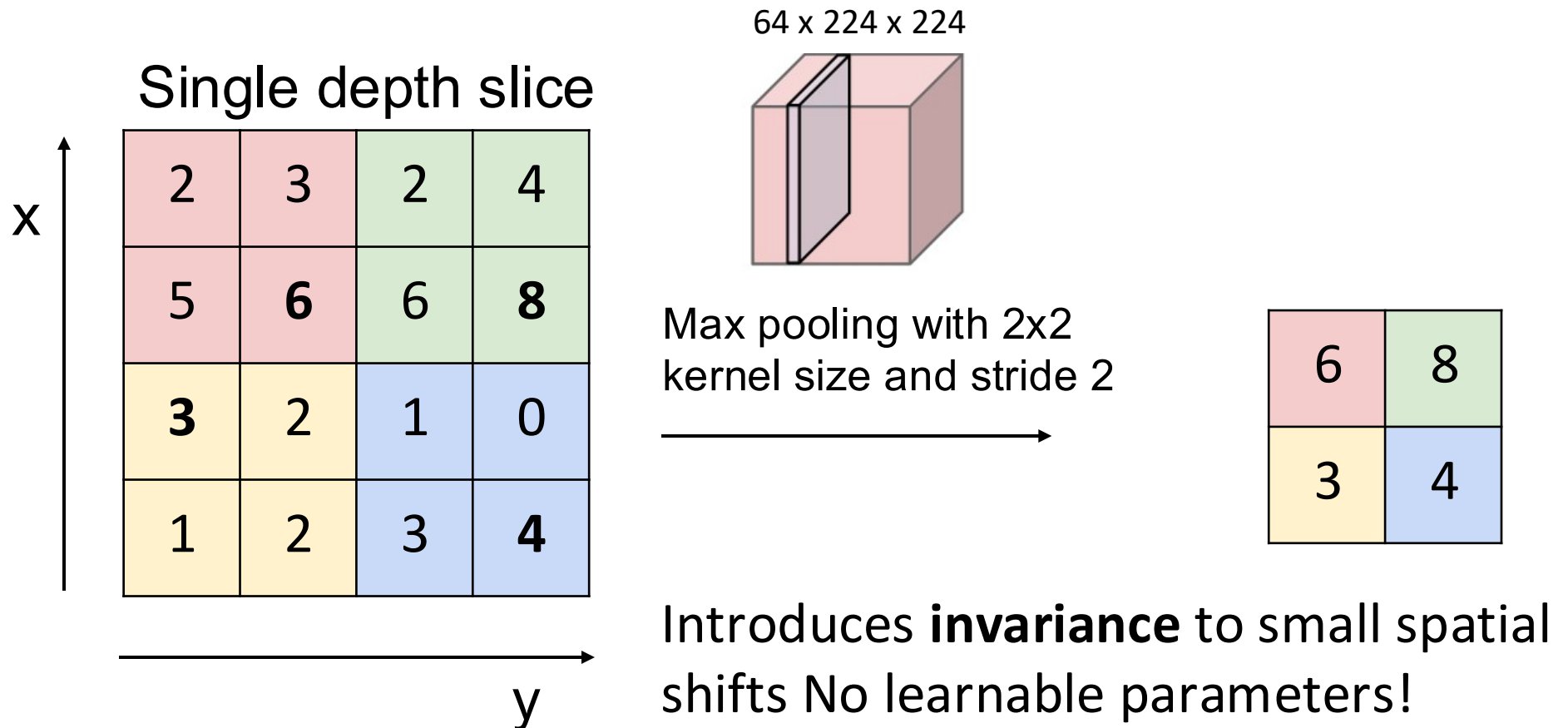
Once you understand how convolutional layers work, the pooling layers are quite easy to grasp. Their goal is to *subsample* (i.e., shrink) the input image in order to reduce the computational load, the memory usage, and the number of parameters (thereby limiting the risk of overfitting).



Hyperparameters:
Kernel Size
Stride
Pooling function

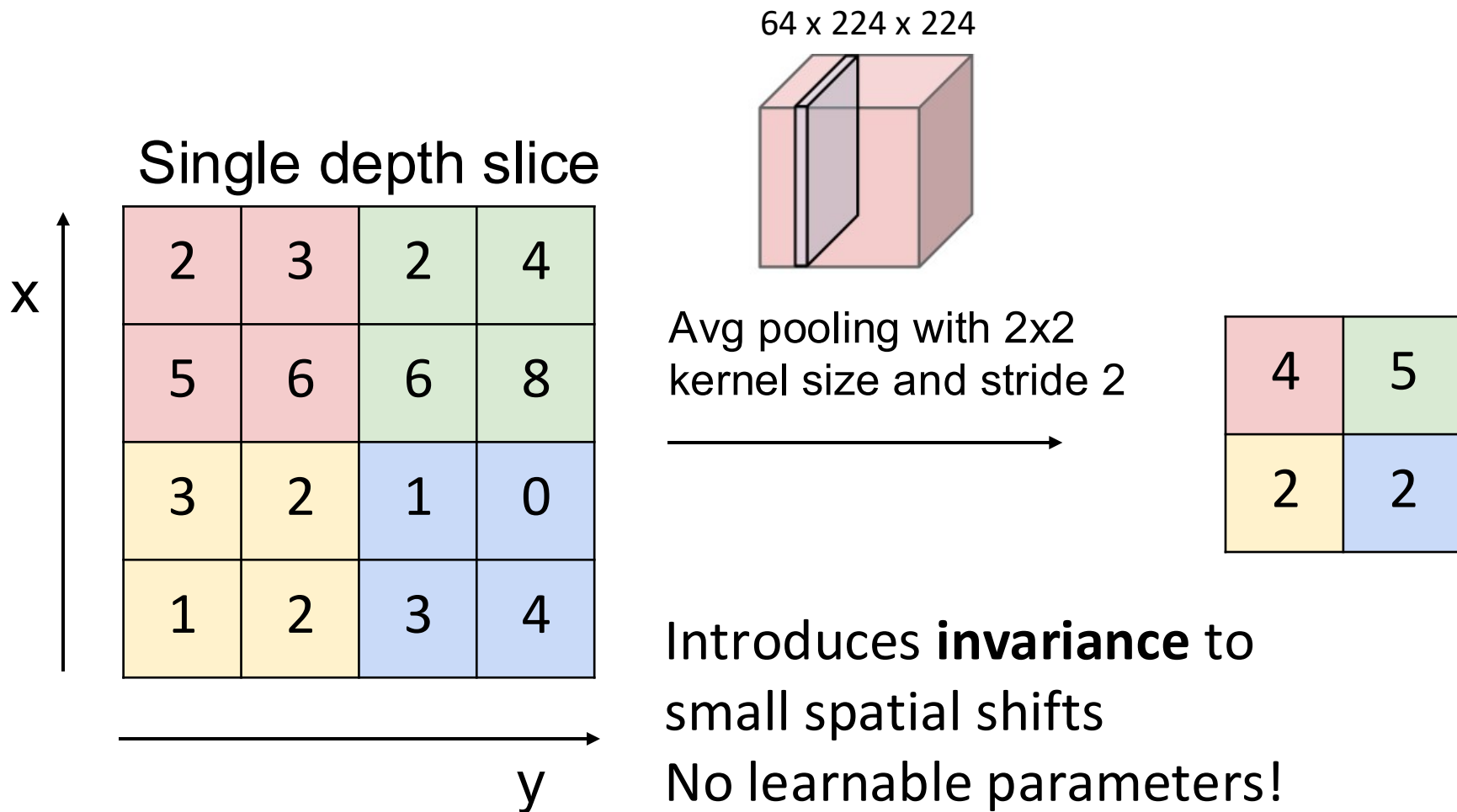
Max Pooling

`Max_pool = keras.layers.MaxPool2D(pool_size=2)`



Average Pooling

```
global_avg_pool = keras.layers.GlobalAvgPool2D()
```



Pooling Summary

Input: $C \times H \times W$

Hyperparameters:

- Kernel size: K
- Stride: S
- Pooling function (max, avg)

Common settings:

max, $K = 2$, $S = 2$

max, $K = 3$, $S = 2$ (AlexNet)

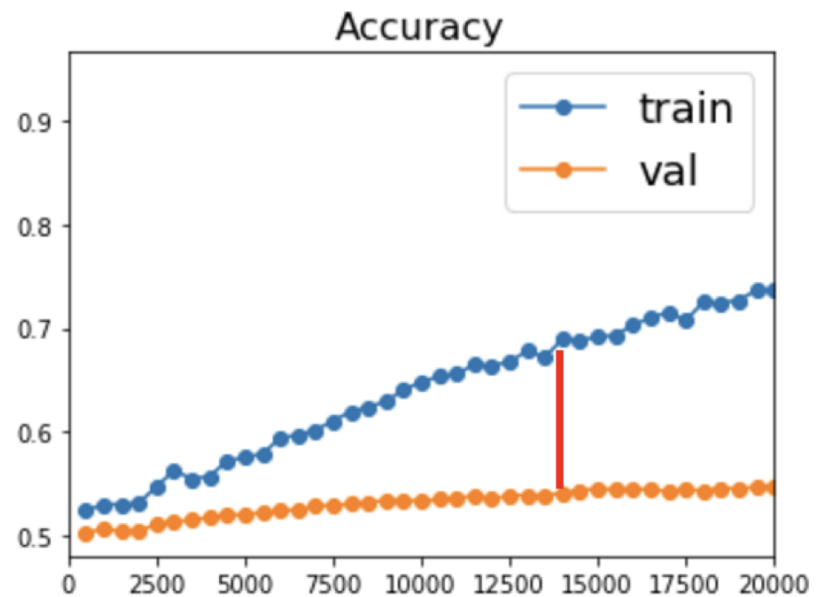
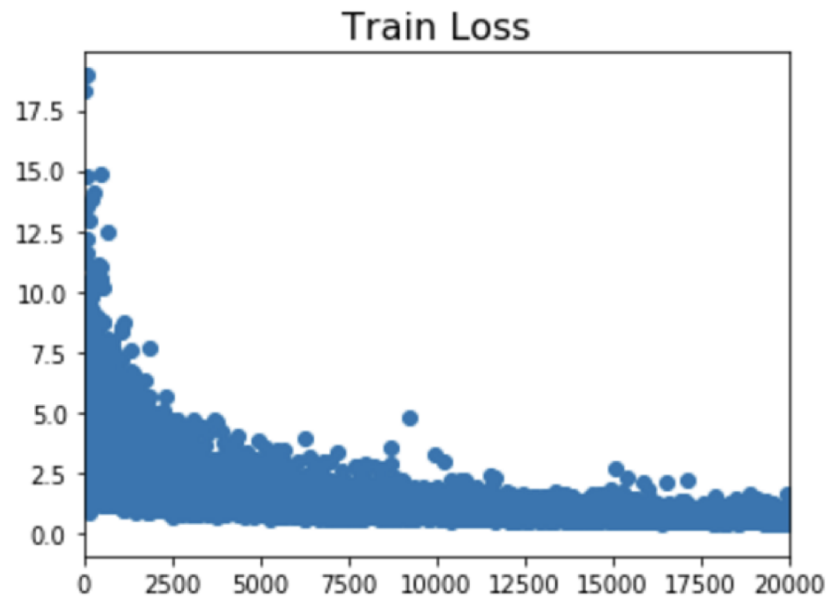
Output: $C \times H' \times W'$ where

- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

Learnable parameters:

None!

**Now your model is training ...
but it overfits!**



Regularization

Regularization: Add term to the loss

$$L = \frac{1}{N} \sum_{i=1}^N \sum_{j \neq y_i} \max(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1) + \lambda R(W)$$

In common use:

L2 regularization

$$R(W) = \sum_k \sum_l W_{k,l}^2 \quad (\text{Weight decay})$$

L1 regularization

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

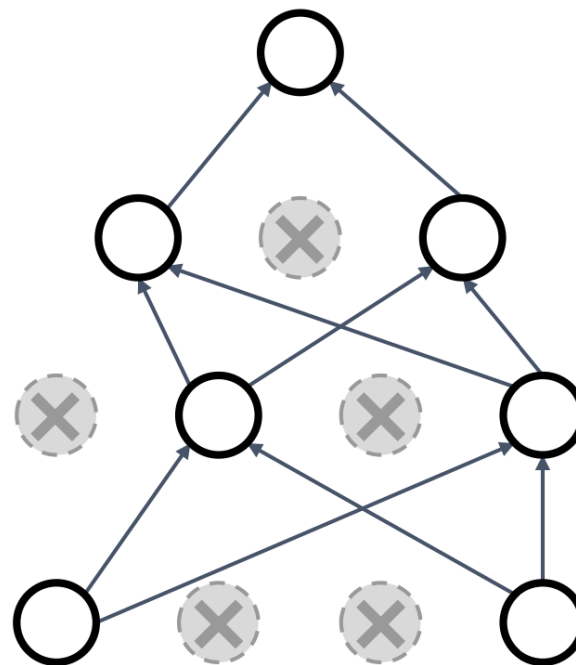
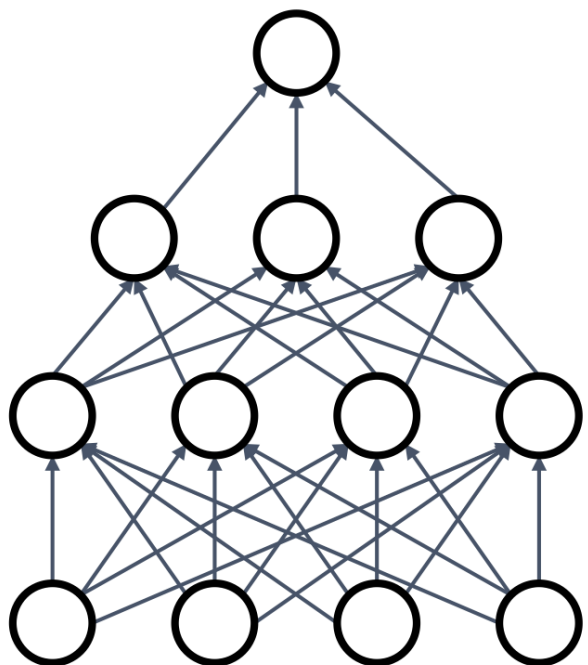
Elastic net (L1 + L2)

$$R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$$

```
layer = keras.layers.Dense(100, activation="elu",  
kernel_initializer="he_normal",  
kernel_regularizer=keras.regularizers.l2(0.01))
```

Regularization: Dropout

In each forward pass, randomly set some neurons to zero
Probability of dropping is a hyperparameter; 0.5 is common

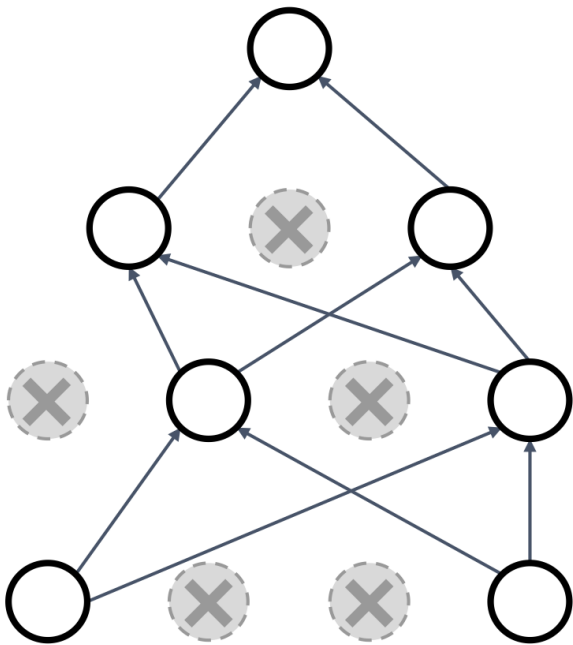


101

Regularization: Dropout

How can this possibly be a good idea?

Forces the network to have a redundant representation; Prevents **co-adaptation** of features



Layers – Dropout

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
```

If you observe that the model is overfitting, you can increase the dropout rate.

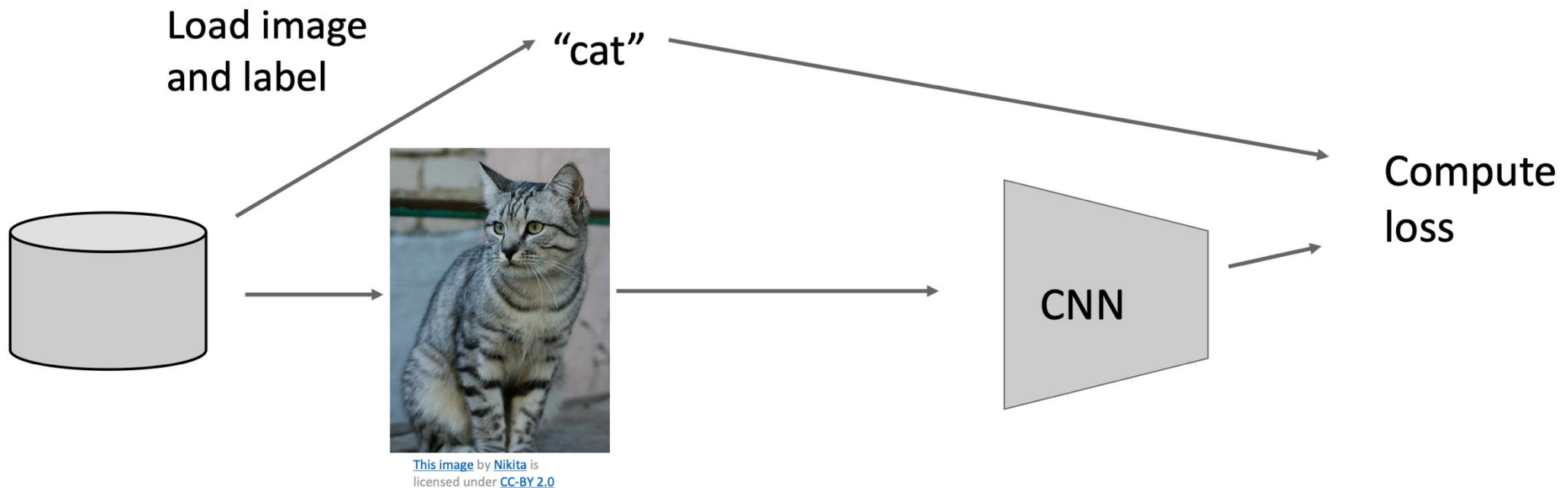
Dropout does tend to significantly slow down convergence, but it usually results in a much better model when tuned properly. So, it is generally well worth the extra time and effort.

Layers – Monte-Carlo (MC) Dropout at Test Time

can average decision probabilities over several random dropout phases to get a better estimate of decision probability on network trained with dropout.

```
with keras.backend.learning_phase_scope(1): # force training mode = dropout on
    y_probas = np.stack([model.predict(X_test_scaled)
                        for sample in range(100)])
y_proba = y_probas.mean(axis=0)
```


Regularization: Data Augmentation



Regularization: Data Augmentation

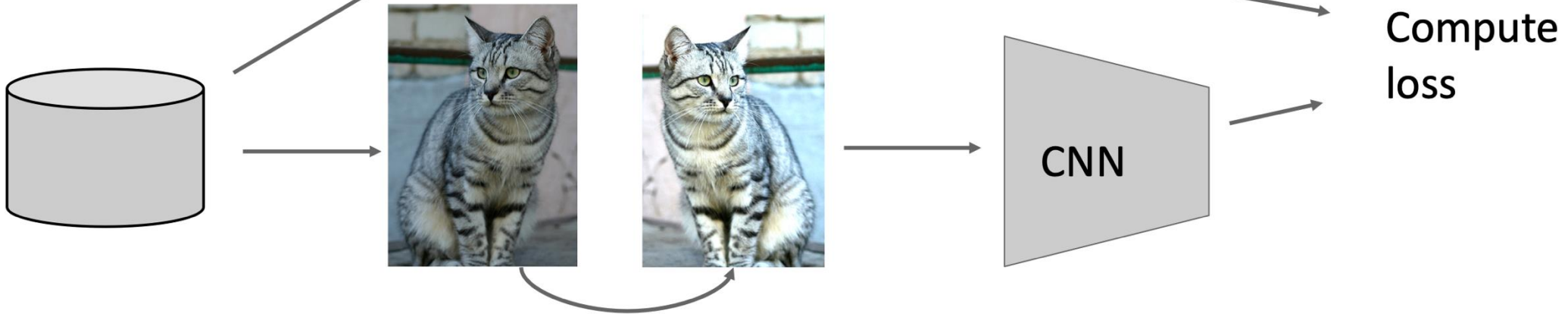
Load image
and label

“cat”

Compute
loss

CNN

Transform image



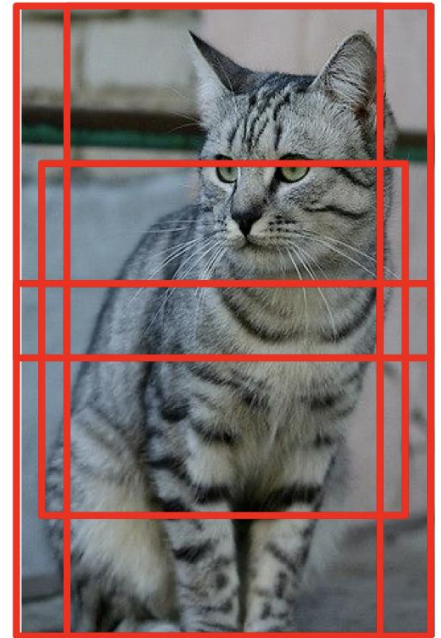
Regularization: Data Augmentation

Horizontal Flips



Regularization: Data Augmentation

Random crops and scales



Regularization: Data Augmentation

Get creative for your problem!

Examples of data augmentations:

- translation
- rotation
- stretching
- shearing,
- lens distortions, ... (go crazy)

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Compilation Stage Options – Overview

```
model.compile(loss="sparse_categorical_crossentropy",  
              optimizer="sgd",  
              metrics=["accuracy"])
```

- **Loss Function** - function of output of network (guessed label) and label, whose average across training is to be minimized by adjusting parameters.
- **Metrics** - easy to understand quantities (like accuracy) that indicate how well the model is doing
- **Optimizer** - numerical method that will be used to attempt to minimize the average loss

Compilation Stage Options - Loss Functions

A loss function tells how good our current model is

Low loss = good model

High loss = bad model

(Also called: objective function; cost function)

Negative loss function
sometimes called reward
function, profit function, utility
function, fitness function, etc

Given a dataset

$$\{(x_i, y_i)\}_{i=1}^N$$

of images x_i and labels y_i ,

Loss for a single example is:

$$L_i(f(x_i, W), y_i)$$

Loss for the dataset is

$$L = \frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)$$

Compilation Stage Options - Loss Functions

<https://keras.io/api/losses/>

Common loss function: Cross-Entropy Loss (Multinomial Logistic Regression)
Want to interpret raw classifier scores as probabilities

Classifier scores
 $s = f(x_i, W)$

Softmax function

$$p_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}$$

Loss

$$L_i = -\log(p_{y_i})$$

1. sparse categorical cross entropy if label y_k is true class index
2. categorical cross entropy if label y_k is one-hot encoded (vector of length of number of classes with 1 in the position of the true class)

Most common for regressor:

- mean squared error

Compilation Stage Options - Metrics

<https://keras.io/api/metrics/>

Accuracy = how often predictions equal labels

TopKCategoricalAccuracy = Computes how often integer targets are in the top K predictions.

```
tf.keras.metrics.SparseTopKCategoricalAccuracy(k=5,  
name="top_k_categorical_accuracy", dtype=None)
```

Can pass a list of several different metrics to compile command (e.g. several k values)
will discuss more later.

Compilation Stage Options - Optimizers

<https://keras.io/api/optimizers/>

Mostly will use SGD and Adam.

1. Momentum

- Speeds up SGD by adding a fraction of the past update to the current one.
- Helps accelerate gradients in the right direction and dampens oscillations.

- **Update rule:**

$$v_t = \gamma v_{t-1} + \eta \nabla J(\theta)$$

$$\theta = \theta - v_t$$

where γ is momentum (typically 0.9)

2. Adagrad

- Adapts the learning rate to each parameter based on the past gradients.
- Effective for sparse data.
- Learning rate decreases over time.

3. RMSProp

- Improves Adagrad by using a moving average of squared gradients.
- Keeps learning rate from decaying too quickly.
- Often used in RNNs.

4. Adam (Adaptive Moment Estimation)

- Combines Momentum and RMSProp.
- Maintains both a moving average of the gradient and its square.
- Very popular and robust for most deep learning tasks.

beta_1:

- Controls the **decay rate of the moving average of the gradient (momentum)**.
- Typical value: **0.9**
- Closer to 1 → smoother average (slower to react)
- Lower value → faster adaptation, but more noise

from tensor

Momentum

optimizer =

beta_2:

- Controls the **decay rate of the moving average of the squared gradient** (like `rho` in RMSProp).
- Typical value: **0.999**
- Helps in scaling learning rates for each parameter
- Closer to 1 → smoother average (slower to react)

#

op

#

op

- Lower value → faster adaptation, but more noise
- Controls the **decay rate** of the moving average of squared gradients.
- Value: typically **0.9**

#

op

- **High** `rho` → smoother average (slower adaptation to new gradient information)
- **Low** `rho` → faster response to recent changes, but more fluctuation

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Arguments to model.fit

- can additionally track validation loss (beyond training loss) during optimization by providing validation data
- providing callback enable the model to
 - be periodically saved (checkpoint)
 - halt training with early stopping

```
history = model.fit(X_train, y_train, epochs=100,  
                    validation_data=(X_valid, y_valid),  
                    callbacks=[cba, cbb])
```

Arguments to model.fit - Callbacks

checkpoint – Model is saved at the end of every epoch, if it's the best seen so far.

```
checkpoint_cb= keras.callbacks.ModelCheckpoint("my_keras_model.h5", save_best_only=True)
history = model.fit(X_train, y_train, epochs=10,
                    validation_data=(X_valid, y_valid),
                    callbacks=[checkpoint_cb])
model = keras.models.load_model("my_keras_model.h5") # rollback to best model
mse_test = model.evaluate(X_test, y_test)
```

EarlyStopping – stops after validation see no progress for patience epochs.

```
model.compile(loss="mse", optimizer=keras.optimizers.SGD(lr=1e-3))
early_stopping_cb = keras.callbacks.EarlyStopping(patience=10,
                                                  restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100,
                    validation_data=(X_valid, y_valid),
                    callbacks=[checkpoint_cb, early_stopping_cb])
mse_test = model.evaluate(X_test, y_test)
```


Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Saving and Reloading Models

- Call `tf.keras.Model.save` to save a model's architecture, weights, and training configuration in a single file/folder.
- Entire model can be saved in 3 file formats (.keras, SavedModel and HDF5).
- This allows you to export a model so it can be used without access to the original Python code.
 - Saving custom objects?
https://www.tensorflow.org/tutorials/keras/save_and_load
 - need to include them if .keras or .h5
- Since the optimizer-state is recovered, you can resume training from where left off.
- .keras default now (2.15), SavedModel format was default in TF2.13. Can choose.

saved model format

```
model.save('myModel')
```

```
new_model=tf.keras.models.load_model('myModel')
```

hdf5 format

```
model.save('myModel.h5')
```

```
new_model=tf.keras.models.load_model('myModel.h5')
```

Overview – Beginner's Keras on TensorFlow

1. Review Keras Model Workflow

- (a) create the model
- (b) compile the model (loss, optimizer, metrics)
- (c) fit the model (checkpoint & early stopping callbacks)
- (d) evaluate the model

2. Model Creation Options

- (a) 2 Methods for Composing Layers
- (b) Layer Options (dense, conv, dropout, max/avg-pooling, concatenate)

3. Compilation-Stage Options

- (a) Selecting a Loss Function & Metrics to Track
- (b) Optimizer Options

4. Arguments to Fit

- (a) Including a Validation Set
- (b) Useful Callbacks (checkpoint & early stopping)

5. Saving and Reloading Models

6. Deep Learning Major Benefit - Transfer Learning

Transfer Learning

**“You need a lot of a data if you
want to train/use CNNs”**

Transfer Learning

“You need a lot of a data if you want to train/use CNNs”

BUSINESS

Transfer Learning with Keras

- one appeal of deep nets is that they easily admit transfer
- take lower layers of a network good at one task (e.g. object recognition) and use them in a similar task (e.g. flower recognition)

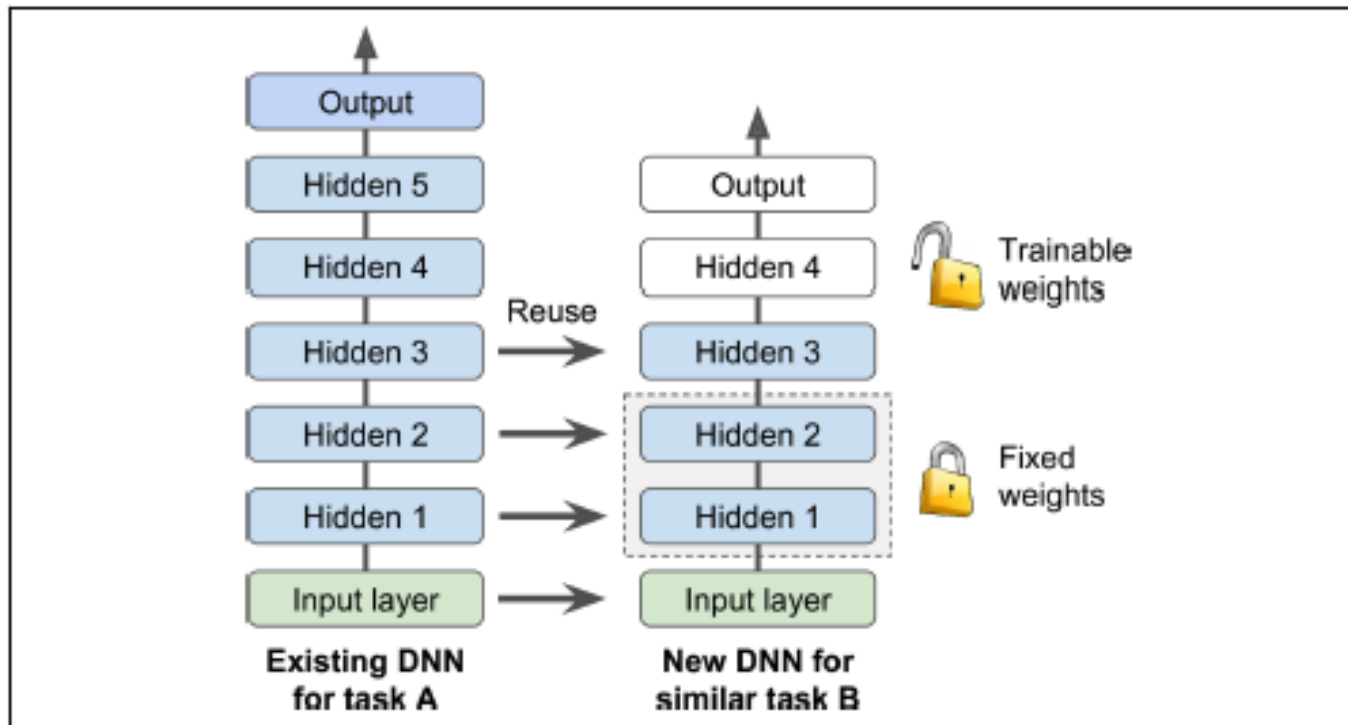
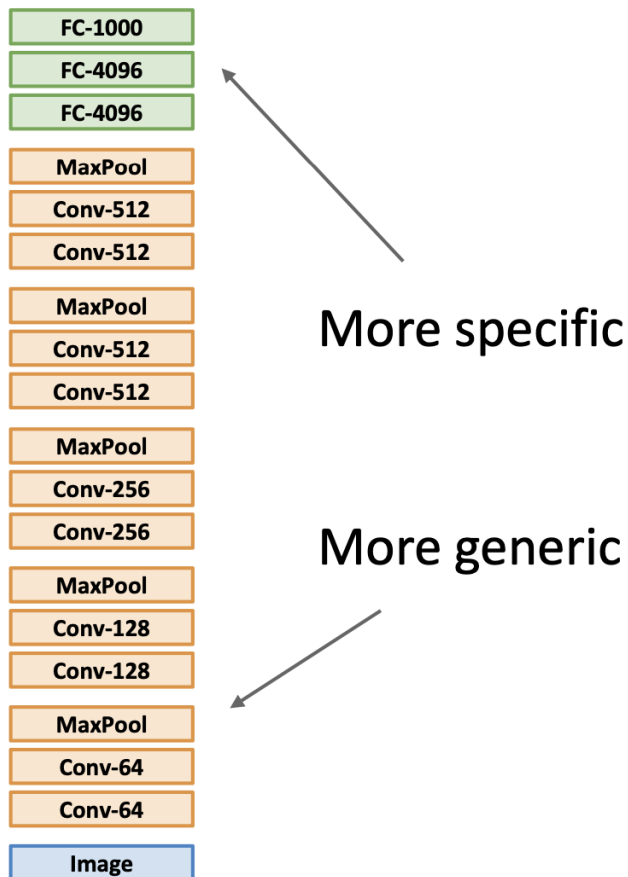


Figure 11-4. Reusing pretrained layers



If the input pictures of your new task don't have the same size as the ones used in the original task, you will usually have to add a preprocessing step to resize them to the size expected by the original model. More generally, transfer learning will work best when the inputs have similar low-level features.

Transfer Learning with CNNs



	Dataset similar to ImageNet	Dataset very different from ImageNet
very little data (10s to 100s)	Use Linear Classifier on top layer	You're in trouble... Try linear classifier from different stages
quite a lot of data (100s to 1000s)	Finetune a few layers	Finetune a larger number of layers

Transfer Learning with Keras

First, you need to load model A, and create a new model based on the model A's layers. Let's reuse all layers except for the output layer:

```
model_A = keras.models.load_model("my_model_A.h5")
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))
```

Note that `model_A` and `model_B_on_A` now share some layers. When you train `model_B_on_A`, it will also affect `model_A`. If you want to avoid that, you need to clone `model_A` before you reuse its layers. To do this, you must clone model A's architecture, then copy its weights (since `clone_model()` does not clone the weights):

```
model_A_clone = keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

start off freezing the weights in the lower layers...

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False
```

recompile to cause this freeze

```
model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd",
                     metrics=["accuracy"])
```


Transfer Learning with Keras

after frozen for a few epochs, can resume adapting them again too

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,  
                           validation_data=(X_valid_B, y_valid_B))  
  
for layer in model_B_on_A.layers[:-1]:  
    layer.trainable = True  
  
optimizer = keras.optimizers.SGD(lr=1e-4) # the default lr is 1e-3  
model_B_on_A.compile(loss="binary_crossentropy", optimizer=optimizer,  
                     metrics=["accuracy"])  
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,  
                           validation_data=(X_valid_B, y_valid_B))
```

more detailed example demonstrating this practically after we review basic imagenet object recognition models

Overview – TensorFlow - Graph Basics, Data API, Preprocessing

1. Convenient Dataset Commands

(a) batch, filter, take, map, shuffle, cache, prefetch

2. Converting your data to easily load into Data API w/ tfrecords

3. Using Keras Preprocessing Layers to Handle Simple Data Processing

4. How to Work with Heterogeneous Data

(a) Datasets of Dictionaries

i. Saving Them

ii. Reloading them & Preprocessing

iii. Making a Keras Model For them

7. Customizing Keras

(a) Custom Loss Functions

(b) Custom Layer

Convenient Data API Dataset Commands

1. batch - group data into blocks (batches) of same size
2. filter - keep only the part of the data that returns true for the given filter/test function
3. take - return k instances from the dataset
4. map - apply a function to each element of the dataset
5. shuffle - buffer shuffle the dataset
6. prefetch - always keep the next k elements in memory
7. cache - hold the entire dataset in memory

TensorFlow's Data API – Introduction

TensorFlow's data API makes operations to group, repeat, shuffle, and cache data easy. For example, suppose we want to repeat a data set three times, then group it into batches of size 7

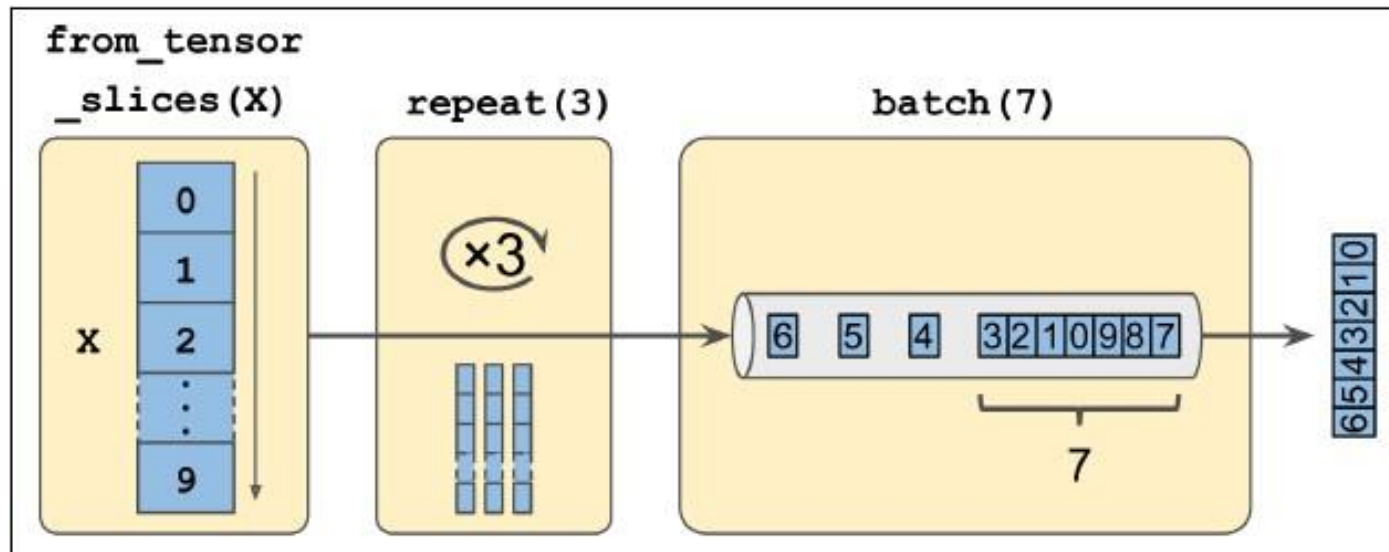


Figure 13-1. Chaining Dataset Transformations

```
>>> X = tf.range(10) # any data tensor
>>> dataset = tf.data.Dataset.from_tensor_slices(X)
>>> dataset

>>> dataset = dataset.repeat(3).batch(7)
>>> for item in dataset:
...     print(item)
```

note, repeat without a number builds an infinite repeating stream!!

TensorFlow's Data API – Mapping, Filtering, Shuffling

Once we have created a dataset, we can transform it elementwise as in

```
>>> dataset = dataset.map(lambda x: x * 2) # Items: [0,2,4,6,8,10,12]
```

Or perform an operation on the whole of it as in

```
>>> dataset = dataset.apply(tf.data.experimental.unbatch()) # Items: 0,2,4,...
```

Or pull out parts of it as in

```
>>> dataset = dataset.filter(lambda x: x < 10) # Items: 0 2 4 6 8 0 2 4 6...
```

A dataset can be buffer-shuffled (keep buffer full, selecting one at random, sending it and replacing it) with

```
>>> dataset = tf.data.Dataset.range(10).repeat(3) # 0 to 9, three times
>>> dataset = dataset.shuffle(buffer_size=5, seed=42).batch(7)
>>> for item in dataset:
...     print(item)
...
tf.Tensor([0 2 3 6 7 9 4], shape=(7,), dtype=int64)
tf.Tensor([5 0 1 1 8 6 5], shape=(7,), dtype=int64)
tf.Tensor([4 8 7 1 2 3 0], shape=(7,), dtype=int64)
tf.Tensor([5 4 2 7 8 9 9], shape=(7,), dtype=int64)
tf.Tensor([3 6], shape=(2,), dtype=int64)
```

TensorFlow's Data API – Pulling Lines from Text Files

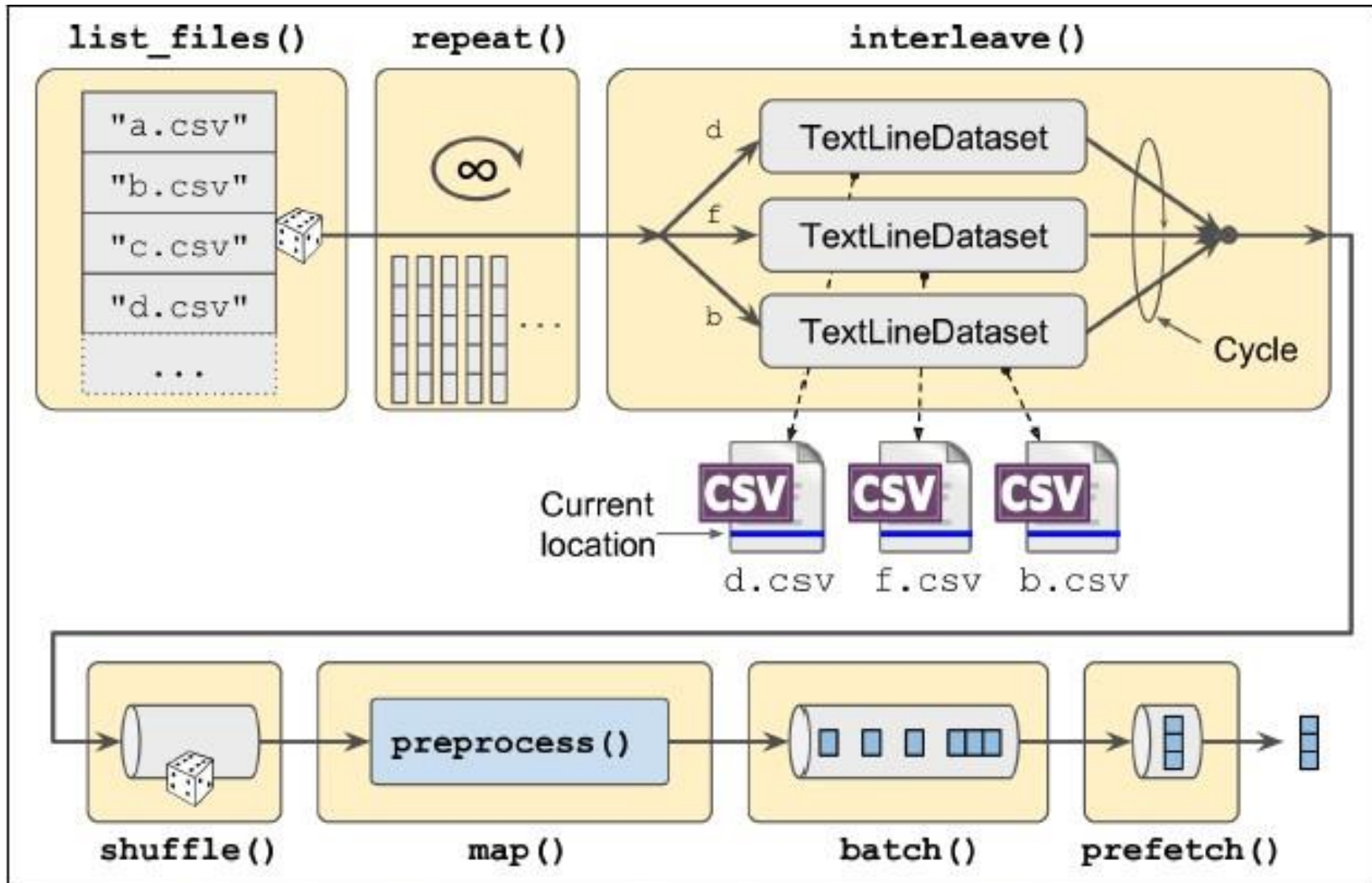


Figure 13-2. Loading and Preprocessing Data From Multiple CSV Files

TensorFlow's Data API – Pulling Lines from Text Files

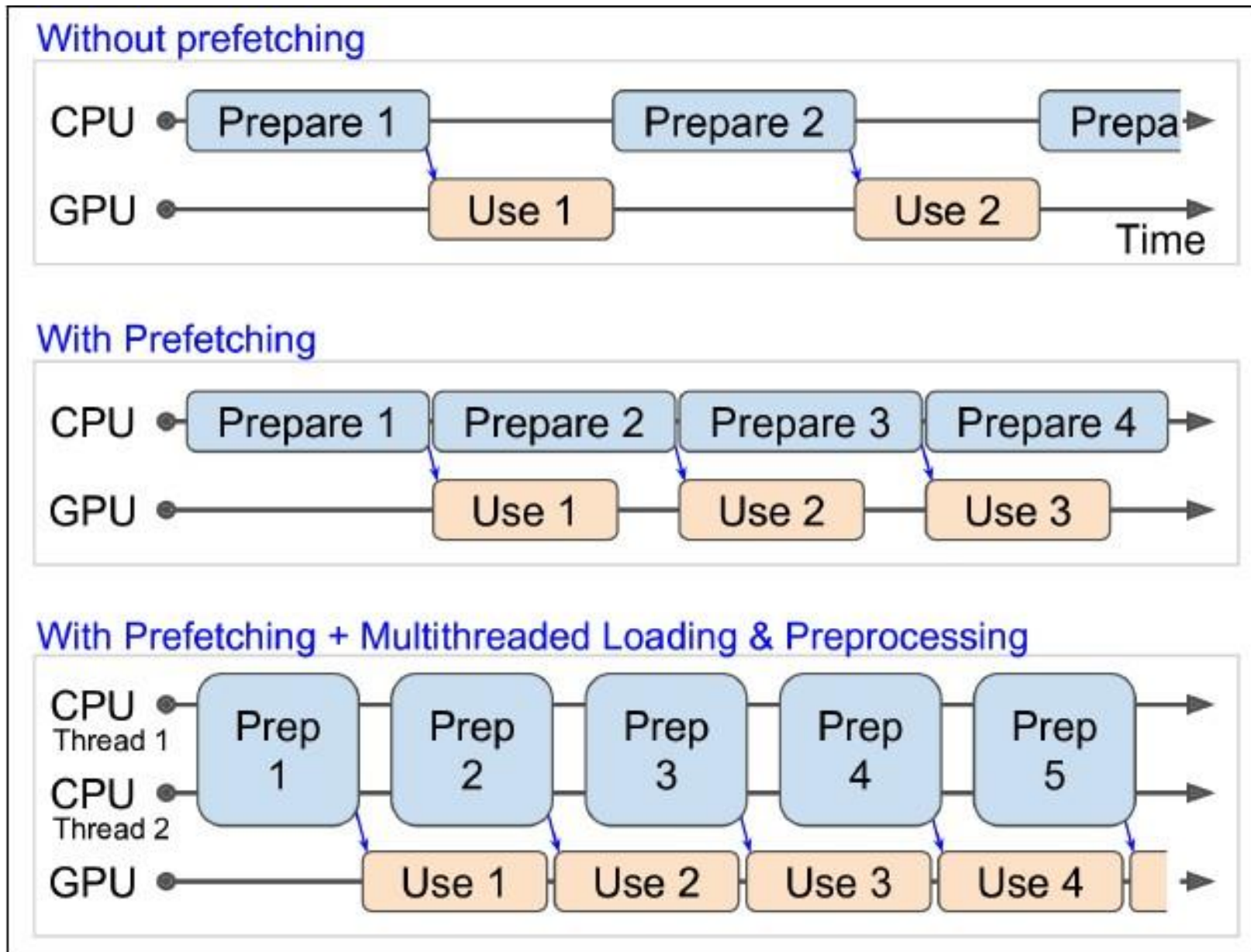


Figure 13-3. Speedup Training Thanks to Prefetching and Multithreading

TensorFlow's Data API – Pulling Lines from Text Files

```
X_mean, X_std = [...] # mean and scale of each feature in the training set
n_inputs = 8
```

```
def preprocess(line):
    defs = [0.] * n_inputs + [tf.constant([], dtype=tf.float32)]
    fields = tf.io.decode_csv(line, record_defaults=defs)
    x = tf.stack(fields[:-1])
    y = tf.stack(fields[-1:])
    return (x - X_mean) / X_std, y

def csv_reader_dataset(filepaths, repeat=None, n_readers=5,
                       n_read_threads=None, shuffle_buffer_size=10000,
                       n_parse_threads=5, batch_size=32):
    dataset = tf.data.Dataset.list_files(filepaths).repeat(repeat)
    dataset = dataset.interleave(
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),
        cycle_length=n_readers, num_parallel_calls=n_read_threads)
    dataset = dataset.shuffle(shuffle_buffer_size)
    dataset = dataset.map(preprocess, num_parallel_calls=n_parse_threads)
    dataset = dataset.batch(batch_size)
    return dataset.prefetch(1)
```